An Explainable AI Pipeline for Academic Integrity Verification

Transparent AI-Text Detection with Argument-Aware Question Generation

Mykhailo Shpyl

M.Sc. in Artificial Intelligence and Big Data Analytics, 2026

Department of Computing, ATU Donegal, Port Road, Letterkenny, Co. Donegal, Ireland.

Working draft for supervisor review, June 2026

An Explainable AI Pipeline for Academic Integrity Verification

Transparent AI-Text Detection with Argument-Aware Question Generation

Author: Mykhailo Shpyl

Supervised by: Dr. Vini Vijayan

A thesis submitted in partial fulfilment of the requirements for the

Master of Science in Artificial Intelligence and Big Data Analytics

Submitted to Atlantic Technological University

Arna chur isteach chuig Ollscoil Teicneolaiochta an Atlantaigh

August 2026

Declaration

I hereby certify that the material, which I now submit for assessment on the programmes of study leading to the award of Master of Science in Artificial Intelligence and Big Data Analytics, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution. I understand that it is my responsibility to ensure that I have adhered to ATU's rules and regulations.

I hereby certify that the material on which I have relied on for the purpose of my assessment is not deemed as personal data under the GDPR Regulations. Personal data is any data from living people that can be identified. Any personal data used for the purpose of my assessment has been pseudonymised and the data set and identifiers are not held by ATU. Alternatively, personal data has been anonymised in line with the Data Protection Commissioner's Guidelines on Anonymisation. All datasets used in this work are publicly available and openly licensed, and no human participants were involved in the study.

I consent that my work will be held for the purposes of education assistance to future students and will be shared on the ATU Donegal (Computing) website (atucomputingdonegal.com) and Research THEA website (https://research.thea.ie/). I understand that documents once uploaded onto the website can be viewed throughout the world and not just in Ireland. Consent can be withdrawn for the publishing of material online by emailing Jade Lyons, Head of Department, at Jade.Lyons@atu.ie to remove items from the ATU Donegal Computing website, and by emailing Denise McCaul, Systems Librarian, at denise.mccaul@atu.ie to remove items from the Research THEA website. Material will continue to appear in printed formats once published, and as websites are a public medium, ATU cannot guarantee that the material has not been saved or downloaded.

Signature of Candidate: Mykhailo Shpyl Date: _____

Acknowledgements

I would like to thank my supervisor, Dr. Vini Vijayan, for her guidance and feedback throughout this project, and the staff of the MSc in Artificial Intelligence and Big Data Analytics at Atlantic Technological University for their support.

Abstract

Universities increasingly rely on automatic detectors to judge whether student work was written with generative AI. Most return a single percentage with nothing behind it. A lecturer cannot defend that number if a student challenges it, and it says nothing about whether the student understands the work they submitted. This dissertation builds a different kind of tool: a pipeline that detects likely AI text, explains each decision in plain language, finds the claims a flagged submission makes, and writes verification questions tied to those claims, so the lecturer can check understanding in a short conversation instead of relying on an accusation. Every stage runs on one consumer laptop, so student work never has to leave the institution.

The detector was trained on 640 human essays from the British Academic Written English corpus paired with 640 length-matched and topic-matched AI essays generated locally. A first version scored perfectly, and that score turned out to be false: an audit traced it to formatting markup leaked into the corpus, and after cleaning, the honest F1 is 0.99. The cleaned model catches 100 percent of test essays written by two commercial generators it never saw in training, without falsely flagging any of the matching human essays. On human writing from unfamiliar domains its false-positive rate rises sharply, so a hybrid version fuses the transformer with stylometric features and cuts those false flags three to eight times. The explanations behind each flag are tested rather than assumed: token-level attribution methods fail a faithfulness test that feature-level SHAP passes, so the feature account is the one a lecturer sees.

Question quality is measured with a judge-free simulation: a question is worth asking if a model that has read the essay answers it much better than a model that has not. The main comparison holds the claim set fixed across four question writers and thirty essays, 901 questions in all. A QLoRA fine-tune of a 3B model, trained on one 8 GB laptop, writes the best questions: it more than doubles its own base model ($p < 0.0001$) and beats the free commercial tier on 24 of the 29 essays both cover ($p = 0.0003$), with every question well-formed. How that result was reached is part of the contribution. An earlier fine-tune's apparent win was an artifact of degenerate output gaming the metric and was retracted, and a three-judge LLM panel, rerun at sixty questions, ranks questions against the objective measure, so neither judge ratings nor raw scores are trusted unchecked anywhere in this

pipeline. This document is a working draft for supervisor review and will be revised before final submission.

Acronyms

Acronym	Definition
AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
GenAI	Generative Artificial Intelligence
BAWE	British Academic Written English (corpus)
TTR	Type-Token Ratio
POS	Part of Speech
BIO	Begin-Inside-Outside (sequence labelling scheme)
QLoRA	Quantised Low-Rank Adaptation
SHAP	SHapley Additive exPlanations
IG	Integrated Gradients
FPR	False-Positive Rate
HPC	High-Performance Computing

Table of Contents

Update this field in Word (select all, then press F9) to populate page numbers.

Table of Figures

Chapter 1: Introduction

1.1 Overview

This dissertation builds an explainable pipeline that helps a lecturer check the integrity of a student submission without having to treat a detector's score as the final word. The pipeline does two connected things. It decides whether a piece of writing is likely AI-generated, and rather than returning a bare percentage it shows which features of the writing drove that decision. Then, for a flagged submission, it reads the student's own argument, pulls out the specific claims they made, and turns those claims into verification questions a lecturer can ask in a short conversation. The aim is to give the lecturer evidence they can explain and defend, tied to the student's own words, so a fair decision has something to rest on. Catching students out is not the goal.

Every stage is meant to be explainable, so a lecturer can see and justify why a submission was flagged and where each question came from, instead of pointing at a number they cannot account for. The question generation is also built twice, once with a commercial large language model and once with a smaller open model that runs on a single laptop. Building it twice lets the project measure how close a locally run, low-cost model can get to an expensive commercial one for this task (Oketch et al., 2025). The question matters to any institution, and the comparison is a central contribution of this dissertation.

I built the system for a lecturer on a large module, somewhere between 150 and 300 students, who has neither the time to prepare a fair set of verification questions for every flagged essay nor a good way to defend a black-box score if a student challenges it. The system has to save that lecturer time. It also has to treat students fairly, including students who do not write in English as a first language and whom existing detectors tend to penalise (Liang et al., 2023), and it has to produce something the lecturer can stand behind.

1.2 Background and context

Over the last few years generative AI tools have moved from a curiosity to something most students have used at least once (Jin et al., 2024). Drafting an opening paragraph, rephrasing an awkward sentence, or getting a hard idea explained now takes a few seconds of typing into a chatbot. That is genuinely useful for learning. It also puts pressure on a basic academic question, whether a piece of work is the student's own and reflects their own understanding.

Universities responded quickly, and a market of AI-text detectors appeared almost as fast as the writing tools did (Wu et al., 2025). Broadly the detectors fall into two families. Zero-shot or statistical methods use a language model's own probabilities to judge how predictable a passage is, on the assumption that machine text sits in smoother, more probable regions than human text (Mitchell et al., 2023). Supervised methods fine-tune a transformer on labelled human and AI examples and then classify new text (He et al., 2021). Commercial services usually combine such methods behind an interface and return a single figure, an AI-likelihood percentage.

These tools do work some of the time. But they were built fast and they are opaque. Their accuracy drops sharply on text they were not trained for, and simple paraphrasing can evade them (Krishna et al., 2023). More seriously for a university, they hand back a number with nothing behind it, and there is growing evidence that the errors are not spread evenly. Writing by non-native English speakers gets flagged as AI far more often than it should be (Liang et al., 2023). A confident score with no explanation behind it and a known bias in its errors is a poor basis for an academic-integrity decision.

1.3 Problem statement

Current detectors give a lecturer a percentage and stop there. Someone who sees "82 percent AI" has no way to explain that number if the student pushes back, and no real way to know whether it is even right for this particular essay. Accusing a person from that position is weak and risky, and the cost of being wrong falls on the student, whose record and standing are at stake. A bare percentage is not enough to carry a decision with those stakes.

On top of the opacity there is a fairness problem. Because the errors are biased, the students most likely to be wrongly flagged are often those who are already disadvantaged, including people writing in a second language (Liang et al., 2023). A system that repeats that bias and cannot show its reasoning is worse than no system at all.

Even a correct flag does not answer the question a lecturer actually cares about. Knowing that text is probably AI-generated does not tell you whether the student understands the material they handed in. One student might have used a tool heavily and still grasp the argument. Another might have written every word themselves and understood little. A flag is a suspicion about how a text was produced, and it carries no evidence about what the student knows. A

lecturer needs a fair way to move from that suspicion to a check of understanding, and no detector provides one.

This project takes on that distance. It makes the detection step transparent and defensible, then turns a flag into specific, source-grounded questions that test whether the student can account for their own work.

1.4 Motivation

I am aiming this at the lecturer who runs a large module and has no time to sit with each flagged essay and work out a fair way to check it. The lecturer needs a process they can defend. It should treat students fairly, and it has to save time rather than add to the pile. The fairness side matters to me in particular. The research already shows these detectors can be biased against people who do not write in English as a first language (Liang et al., 2023), and I do not want to build something that quietly repeats that. If the system shows its reasoning, its mistakes can be seen and argued with, and a fair process needs that to be possible.

1.5 The verification gap

There is a gap between detection and the question a lecturer actually needs answered. Detection is improving but stays a black box, and whether the student understands their own submission is a separate matter. Nothing I found connects the two, and nothing takes a flag and turns it into specific questions, drawn from the student's own claims, that a lecturer can ask in a short conversation to see whether the understanding is there. This project is built to fill that gap.

1.6 Research question and objectives

The main question is how an explainable AI pipeline can be designed to support lecturers in academic integrity verification, by combining transparent AI text detection with argument-aware question generation, and how far locally fine-tuned open-source models can match commercial ones at producing interpretable, defensible outputs for this task.

To get there the objectives are roughly:

- detect AI-written text in a way that can be explained as well as scored;

- show which words and features drove each decision, and test that those explanations are faithful;
- pull out the claims and evidence in a flagged essay and tie each one to its source;
- generate verification questions from those claims, and compare a commercial model against a local open-source one;
- label each question by cognitive level as a quality check; and
- evaluate the whole thing with objective measures, not opinion.

1.7 Scope and boundaries

To keep this doable in the time I have, the scope is deliberately tight. The detector works with two classes only, human and AI. I dropped the earlier idea of partial-AI classes because they overlap too much to separate reliably; if the two-class version works well, they may come back as future work. The pipeline has six parts (detection, explainability, argument mining, question generation, a Bloom's level check, and the output guide), and the datasets are fixed in advance. The study has no human participants, which keeps it clear of needing ethics approval and keeps the focus on the system itself.

1.8 Contributions

What I think this project adds:

- a pipeline that carries a detection flag through to evidence a lecturer can inspect and defend, instead of stopping at a score;
- verification questions tied back to the student's own writing, so a lecturer can see where each one came from;
- a direct comparison of a locally run open model against a commercial one for this task, relevant to any institution weighing cost and control;
- an objective way to measure whether a question really tests understanding; and
- an examination of how the detector treats native versus non-native writers.

1.9 Dissertation outline

The rest of the document is laid out as follows.

Chapter 2 reviews the recent literature the project draws on, covering AI-text detection and its benchmarks, stylometric features, explainability and faithfulness, argument mining, question generation and its evaluation, and the fairness evidence behind the design.

Chapter 3 sets out the detection methodology and the first results, including the dataset, the construction of the matched AI essays, and the audit that found and removed a corpus artefact behind an initially perfect score.

Chapter 4 describes the implementation of the components built so far, with the engineering decisions that the 8 GB laptop budget forced.

Chapter 5 explains the detector's decisions and tests whether those explanations are faithful, comparing token-level attributions on the transformer against SHAP over the stylometric features.

Chapter 6 tests robustness, looking at how the detector transfers to generators it never saw and to other kinds of text, and what its errors mean for fairness.

Chapter 7 presents the core contribution, turning a flag into verification questions drawn from the student's own claims, with sentence-level provenance and a Bloom's level on each question. It builds the first thin slice, then replaces its stand-ins with the trained Bloom's classifier and the trained claim extractor, assembles the lecturer's Verification Interview Guide, and fine-tunes the local backend.

Chapter 8 evaluates the questions. It builds the judge-free discrimination simulation, runs the commercial-versus-local comparison under two designs, validates a three-judge LLM panel against the objective measure, and reports the fine-tuning data-format experiment together with the quality audit that caught a metric-gaming artefact.

Chapter 9 discusses what the results mean, covering the answer to the research question, the methodological lesson drawn from the negative results, the case for verification over accusation, the limitations, and the implications for practice. Chapter 10 concludes and sets out future work, and the references follow.

Chapter 2: Literature Review

2.1 Scope of the review

This review covers the parts of the literature the project draws on and, in places, argues against: detection of AI-generated text, the stylometric features behind the detector, explainability and faithfulness testing, argument mining, question generation, evaluation of question quality, and fairness. The sections follow the order of the pipeline components and end with the gap in the literature. I anchored the search on the methods and datasets already chosen (the DeBERTa detector, the M4 benchmark, the Persuasive Essays corpus, Bloom's taxonomy, the answerability idea behind the evaluation) and then branched out to alternatives and criticisms.

2.2 Detecting AI-generated text

Work on detecting machine-generated text falls into two broad families. A recent survey by Wu et al. (2025) organises the field along these lines and adds watermarking and human-assisted methods.

The first family is zero-shot and statistical. These methods score text using a language model's own probabilities and need no trained classifier. The clearest example is DetectGPT (Mitchell et al., 2023), which observes that machine-generated passages tend to occupy negative-curvature regions of a model's log-probability surface, and turns that observation into a curvature test requiring no labelled data. Nothing has to be trained. But the signal is subtle and can be disturbed.

The second family is supervised. A pretrained transformer is fine-tuned on labelled human and AI examples and then classifies new text. I chose DeBERTa (He et al., 2021) as the primary detector because its disentangled attention mechanism improves on earlier encoders, with RoBERTa (Liu et al., 2019) as a comparison. Supervised detectors are accurate in-domain but depend heavily on what they were trained on.

Progress is measured on shared benchmarks. The M4 corpus (Wang et al., 2023) collects human and machine text across several generators, domains, and languages, and the SemEval-2024 Task 8 shared task built on it (Wang et al., 2024). I use that benchmark to test

whether a detector trained on one generator and one kind of text transfers to unseen generators and unseen domains.

Both approaches have weaknesses that matter for this project. Detectors are brittle: Krishna et al. (2023) show that a strong paraphraser drops DetectGPT's accuracy from 70.3% to 4.6% without changing the meaning, and accuracy also falls under domain shift. The problem that matters more here is opacity. Commercial detectors return a single score with nothing behind it, and a number without reasons is a weak basis for an academic-integrity decision. This project starts from that problem.

2.3 Stylometric features

Alongside the transformer, the detector uses stylometric features. These describe how a text is written, independent of its subject: perplexity or predictability under a language model, burstiness (variation in sentence length and structure), type-token ratio, sentence-length variance, and part-of-speech distributions. Fluent machine text tends to be smoother and less varied than human writing, and each feature is meant to capture one form of that difference.

There is direct evidence that these surface properties carry a real signal, and sometimes a harmful one. Liang et al. (2023) attribute the misclassification of non-native English writing to its lower perplexity, that is, its more limited linguistic variability, which makes it look more machine-like to a perplexity-based detector. So the same features that separate the classes can also encode bias. The pipeline uses them as interpretable evidence and checks their behaviour separately. Hand-crafted features (perplexity, readability, error-based and lexical features) have also been shown to classify AI-generated and even AI-rephrased text, alone and alongside neural models (Mindner et al., 2023). I took that result as support for the hybrid design.

2.4 Explainability and faithfulness

The explainability layer is central to the project, so this section covers the attribution methods and the work on testing whether an explanation reflects what the model actually did.

The pipeline applies Integrated Gradients (Sundararajan et al., 2017) to the transformer's tokens. The method attributes a prediction to its input features and satisfies two axioms, Sensitivity and Implementation Invariance. SHAP (Lundberg and Lee, 2017) assigns each

feature a Shapley-value importance; here it runs on the stylometric features, where it gives a lecturer-facing, feature-level explanation. Attention weights are sometimes offered as explanations too, but that use has been criticised (Jain and Wallace, 2019). Partly for that reason, this project does not rely on attention.

An explanation is only useful if it is faithful, meaning the things it highlights actually drove the decision. DeYoung et al. (2020) introduced the ERASER benchmark and the comprehensiveness and sufficiency metrics, which measure faithfulness by removing or keeping the features an explanation calls important and watching whether the prediction moves. This project uses the same kind of ablation test. The test showed the transformer's token-level attributions were diffuse and only weakly faithful, while the stylometric SHAP explanation held up.

2.5 Argument mining

The pipeline needs the claims and evidence in a student's essay before it can generate questions grounded in them. The foundational resource is the Persuasive Essays corpus of Stab and Gurevych (2017), which annotates argument components (claims, premises) and the relations between them, and frames the task as token-level sequence labelling plus relation classification. The argument-mining component here follows that setup. More recent work applies transformer ensembles, and even LLM refinement, to argument component classification across several corpora (Pietron et al., 2024).

For this project the extraction also has to carry provenance. Each extracted claim points back to the exact source passage it came from, so a generated question can be tied to where it originated and a lecturer can defend it. Extraction without that link would bring the black-box problem back in at a different stage.

2.6 Automatic question generation

Question generation has moved from neural models that generate a question from a passage to large language models prompted to do the same, with growing interest in controllable and grounded generation (Guo et al., 2024). Grounding matters most for verification. A question is only defensible if it can be traced to a specific claim in the student's own text, so the generator works per extracted claim and records the source sentences.

The comparison at the centre of this project, a commercial model against a locally run open model, sits inside a wider question about cost and control in LLM deployment. In the closest study to this setting, Oketch et al. (2025) compared closed and open LLMs for automated essay scoring and found open models such as Llama 3 and Qwen2.5 comparable to GPT-4 in performance and fairness at up to 37 times lower cost. An institution that could run verification on a laptop instead of a paid API would have a cheaper and more controllable option, if the quality holds. The project tests whether it does.

2.7 Bloom's taxonomy and cognitive level

Each generated question is labelled by cognitive level as a quality control. The labels come from the revised Bloom's taxonomy of Anderson and Krathwohl (2001), which orders cognitive processes from Remember and Understand through Apply and Analyse to Evaluate and Create. The taxonomy gives a principled way to describe what a question demands of a student.

There is a directly relevant dataset for the educational side. EduQG (Hadifar et al., 2022) provides 3,397 questions with their source documents, and 903 of them are annotated with a Bloom's cognitive level. I use that labelled subset to train and check the Bloom's classifier. Automatic classification of question cognitive level is an active research task; approaches run from classical classifiers through transformers to large language models (Kumar et al., 2025).

2.8 Evaluating generated questions

The project's main evaluation asks whether a question genuinely tests understanding. The closest prior idea is answerability-based, reference-free evaluation. RQUGE (Mohammadshahi et al., 2023) drops the reference question and instead scores a generated question by whether a question-answering module can answer it from the given context. The discrimination simulation used in this project works in the same spirit. It asks whether a model can answer a question with the source versus without it, and treats the gap as the question's discriminative value.

The supplementary evaluation uses an LLM-as-judge. The approach is increasingly common but has known reliability and bias problems, including position, verbosity, and self-enhancement biases (Zheng et al., 2023), so judge output has to be checked against something independent before it counts. Cross-model agreement is quantified with

Krippendorff's alpha (Hayes and Krippendorff, 2007) and anchored to the objective discrimination results.

2.9 Fairness and bias in detection

Fairness was one of the motivations for this project. Liang et al. (2023) tested seven widely used GPT detectors on TOEFL essays by non-native English writers and found an average false-positive rate of 61.3%, against about 5.1% for essays by native writers. The detectors systematically misclassified competent non-native writing as AI-generated. Chapter 6 measures the same failure mode directly on my own detector and reports it as a main result rather than a footnote. It is also why transparency and defensibility run through the design.

2.10 Summary and the gap

Detection is improving but stays opaque, and on the evidence above it is sometimes unfair. Explainability methods exist, and there are established ways to test whether they are faithful. Argument mining can recover a student's claims with provenance. Question generation and its evaluation are maturing, and Bloom's taxonomy gives a way to grade cognitive level. Nothing in this literature joins the pieces together. No existing work takes a detection flag and produces transparent, source-grounded verification questions that let a lecturer fairly check whether a student understands their own work. This project addresses that gap. It also takes on the practical question of whether a local model can do the job as well as a commercial one.

Chapter 3: AI text detection (methodology and first results)

3.1 What this component has to do

The first stage of the pipeline decides whether a piece of submitted writing is the student's own or was produced by a generative model. Everything downstream depends on it. A bare score is not enough on its own, though, because the lecturer has to be able to defend whatever decision follows from it. I am therefore as interested in why the model decides what it decides as in how often it is right.

3.2 The detection corpus

I needed text that is labelled human or AI and that does not give the answer away for the wrong reason. The human side is a stratified sample of 640 real student essays from BAWE (Alsop and Nesi, 2009; the sampling is described in the sample design note). For the AI side I generated one essay per human essay with Llama 3.1 8B (Grattafiori et al., 2024) running locally, matched on topic, keywords and target length. The length matching is there because a detector faced with AI essays that ran systematically longer or shorter could score well just by counting words, and would learn nothing about style. After generation the lengths line up closely (Pearson r about 0.98 between each AI essay and its human source, mean length ratio about 1.05), so length carries no usable signal in this corpus.

That gives 1,280 essays, 640 human and 640 AI. I split them by student rather than by essay, so nobody appears in both training and test. Without that rule the model could memorise one person's writing instead of learning the general difference between the classes. The split is 438 / 102 / 100 pairs for train / validation / test.

3.3 The detector

For the detection model I fine-tuned two transformers with the same settings and the same student-level splits: DeBERTa-v3-base as the primary model and RoBERTa-base as a comparison, both classifying human (0) against AI (1). I report accuracy, precision, recall, F1 and the confusion matrix, plus a separate false-positive rate for native and non-native English writers, because wrongly flagging a non-native student as AI is the failure mode I most want to avoid. The second half of the hybrid, a stylometric feature model (perplexity, burstiness,

vocabulary richness, part-of-speech mix), is built in Chapter 5, and the two halves are fused in Section 6.7. This chapter covers the transformer half on its own.

3.4 The first result and why it needed checking

The first trained detector scored a perfect F1 of 1.00 on the held-out test set, for both DeBERTa and RoBERTa. A perfect score needed checking before I could use it. This project argues that detection scores must be explainable, and presenting a number I could not explain myself would undercut that. Before relying on the detector I ran an audit (`src/detection/audit_detector.py`) to find out whether the score was real or a shortcut.

3.5 The audit

The audit covered four checks.

Test-set contamination. I checked that no student sits in more than one split, that each human and AI pair stays in the same split, and that no exact-duplicate text crosses from training into test. All three checks came back clean across 394 students, so the model had not simply seen the answers.

A giveaway in the text itself. There is one, and it is the main finding of the audit. The human essays come from the BAWE plain-text export, which keeps structural tags such as `<heading>`, `<fnote>`, `<list>`, `<figure>` and `<quote>`. About 88 percent of the human essays in my sample contain at least one of these tags. None of the AI essays do, because Llama writes plain prose without corpus markup. A rule as simple as "if the text contains a tag, call it human" reaches 92.5 percent accuracy on the test set on its own, with no understanding of language at all. The AI side has a mirror-image problem: Llama sometimes writes markdown (bold, headings, bulleted and numbered lists) that human plain text never has. Both artefacts come from how each half of the corpus was produced, they say nothing about the writing itself, and they sit right at the start of the text where the model's 512-token window sees them. This shortcut accounts for a large part of the perfect score.

Whether anything real survives cleaning. I wrote a normaliser (`src/detection/text_normalize.py`) that strips the BAWE tags from the human side and the markdown from the AI side and flattens the layout, then rebuilt the corpus from the

cleaned text. On the cleaned text a simple TF-IDF and logistic-regression model still separates the two classes at 100 percent on the test set. I treat that 100 percent with some caution, because one residual markup token ("fnote") survived the cleaning into the full model's vocabulary. Tracing it showed why. The cleaning strips the bracketed tags, but 115 of the 640 AI essays contain the bare word "fnote", echoed by the generator from keyword prompts that were extracted from the human source text. In the cleaned corpus "fnote" therefore flips from a human marker to a weak AI one. It is a generation artefact rather than leftover markup, and it does not touch the function-word or stylometric results. To close the question I later ran the promised control, with the bare token stripped from both classes and the same DeBERTa configuration retrained on the same splits (`src/detection/fnote_control.py`, `outputs/fnote_control.json`). The control scores F1 0.995 against the detector of record's 0.990, one test essay of difference, so "fnote" was carrying nothing and the headline stands without it. The cleanest evidence comes from a stricter test. A model that is allowed to use only function words (the, and, because, therefore, and the like, which carry no topic information at all, and no markup) still reaches 99.5 percent. Topic cannot explain that, and neither can a leftover tag. The remaining signal is writing style.

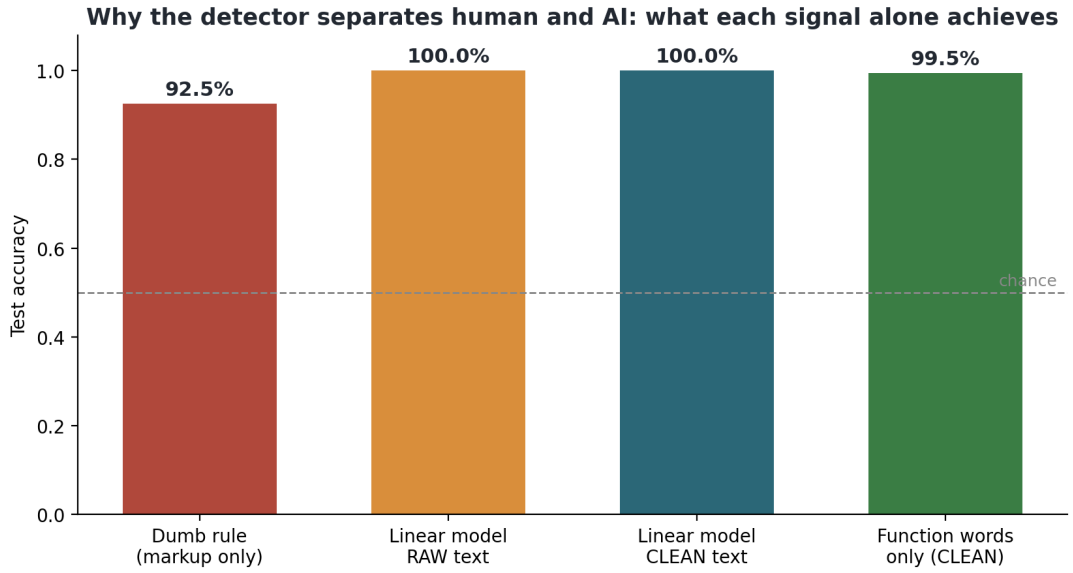


Figure 3.1: What each signal achieves on its own. A markup-only rule already scores 92.5 percent on the raw text, so the artefact is large. After cleaning, a simple model still separates the classes, and a function-words-only model still reaches 99.5 percent; the residual signal is writing style.

What the cleaned model keys on. The cleaned linear model is interpretable, so I can read off the words that push each way. The AI-leaning terms are the familiar large-model register: "in conclusion", "nuanced", "essential", "highlights", "insights", "complex", "significant". The

human-leaning terms are blunter connectives: "therefore", "because", "so", "thus", "very". That matches what the stylometric features already showed, that human writing here varies more and the AI writing is smoother and more uniform.

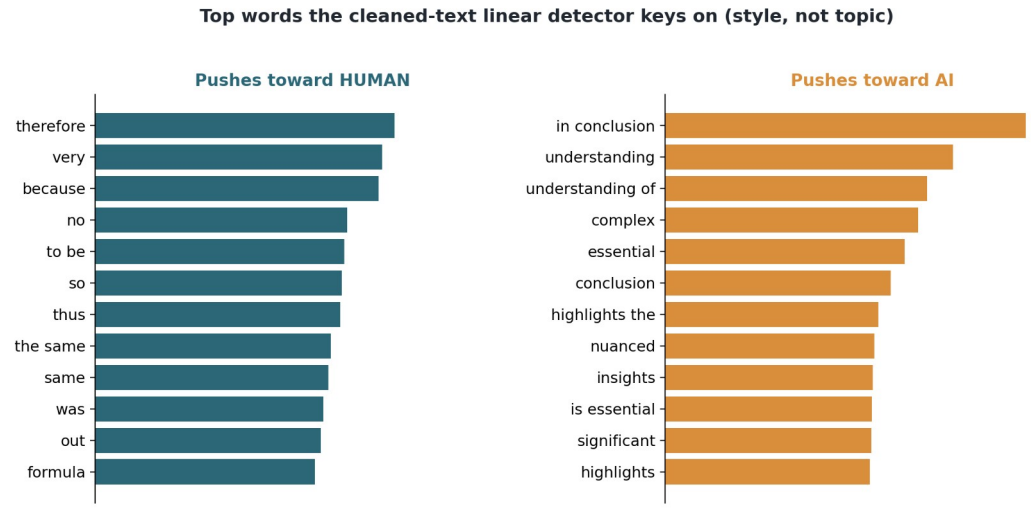


Figure 3.2: The words the cleaned-text model keys on. The AI side is the familiar large-model register; the human side is blunter argument connectives. These word lists feed the explainability layer.

3.6 Results on the cleaned corpus

After removing the artefact I retrained DeBERTa on the cleaned corpus. The score moved off the ceiling, as I had hoped it would. Test accuracy is 0.99, precision 0.98, recall 1.00 and F1 0.990, with a confusion matrix of [[98, 2], [0, 100]]. Two human essays are flagged as AI and no AI essay is missed. The test set is small (100 human and 100 AI essays), so I put a number on that uncertainty. A 95% bootstrap confidence interval on the F1 is about [0.97, 1.00] (`src/evaluation/confidence.py`, Figure 3.5), and one extra misclassification moves the F1 by roughly 0.005, so the headline should be read as "about 0.99" rather than an exact value. The full-document linear probe still scores 1.00 on the same cleaned text while DeBERTa makes two mistakes, which fits the fact that the transformer only reads the opening 512 tokens and the linear model sees the whole essay.

On fairness, the human false-positive rate is 2 of 50 native writers (0.04) and 0 of 50 non-native writers (0.00). With only 50 humans per group, one essay is worth 0.02, so this is not enough to establish a fairness gap, and I treat it as indicative only. The stronger fairness evidence is the cross-domain result in Chapter 6, where the human false-positive rate is measured on far more text and shows the real bias mechanism.

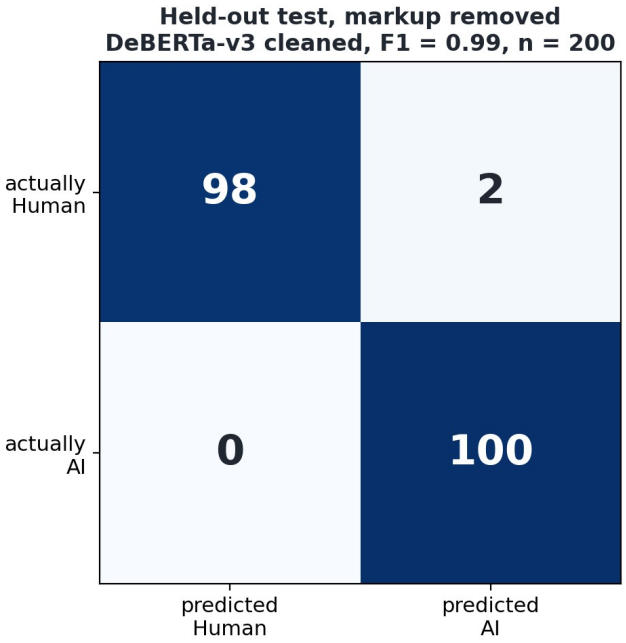


Figure 3.3: DeBERTa on the held-out test set after the markup was removed. Two human essays are flagged as AI and no AI essay is missed (F1 0.990).

RoBERTa, trained the same way on the cleaned corpus, lands in the same place: F1 0.995, a confusion matrix of [[99, 1], [0, 100]], native false-positive rate 0.02. Its only difference from DeBERTa is one human essay flipping, and the two confidence intervals overlap almost completely (Figure 3.5). On this 200-essay test set 0.990 and 0.995 are statistically indistinguishable, so I do not read anything into the gap. Both architectures also make the same kind of mistake: every error is a human essay flagged as AI, and no AI essay is missed. That shared, one-sided error pattern matters more than the near-identical scores, because it indicates that the residual signal is a real style difference and not a quirk of one model.

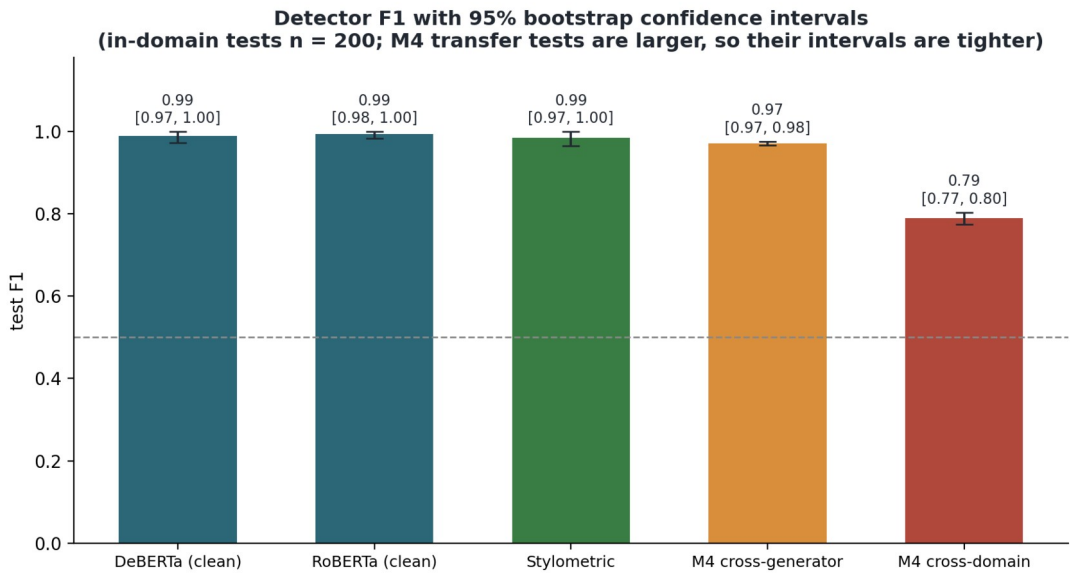


Figure 3.5: Test F1 with 95% bootstrap confidence intervals. The in-domain detectors (DeBERTa, RoBERTa, stylometric) have heavily overlapping intervals on n=200, so their differences are within sampling noise; the M4 transfer results have tight intervals on much larger samples.

The cleaned F1 of 0.990 is the number I report; the raw 1.000 is kept only to show how large the artefact was. Either way the in-domain task is easy, and the literature agrees: separating one known generator from human writing in one domain is close to a solved problem. That easy setting is not where this project makes its contribution. The contribution is the explanation behind each decision and, after this stage, robustness to the harder settings that matter in practice, meaning other generators, paraphrased AI text, and submissions that mix human and AI writing. I expect the score to drop there, and those harder settings are where the real research in this project takes place.

3.7 Why the score is still high after cleaning

A cleaned score of 0.99 could look suspicious in its own right, so I dug into the reasons (`src/detection/why_high.py`) in order to answer the question properly. The score is high because the task as I have built it is the easy version of the problem, and several separate signals reinforce each other.

The biggest reason is that all of my AI text comes from one model with one prompt. The AI class is really "Llama 3.1 writing the way I asked it to", so it has a consistent style. When I measure how similar essays are to each other in style space, the AI essays are more alike (average similarity 0.60) than the human essays (0.56). The humans are a crowd of different people; the AI is one voice repeated 640 times. Separating one consistent voice from a varied

crowd is not hard, and the detection literature treats single-generator, in-domain detection as close to solved.

Several smaller tells sit on top of that, all pointing the same way. The vocabulary differs (the AI register described above). The spelling differs too: the human essays in my sample are in British English (about 2.3 British spellings per thousand words) and Llama defaults to American (about 2.4 American spellings per thousand words). This tell is real but shallow. It is a generator-locale artefact rather than deep evidence of AI authorship, and it would shrink if I prompted the model to write British English, a fix I will try. Formality differs as well: the human essays use about three times as many contractions. None of these on its own is decisive, but they stack. A 2D picture of the essays in pure style space (Figure 3.4, function words only) shows two clouds that barely touch. Given how easy the setting is, 0.99 is the expected result, not evidence of a hidden error.

I checked how much of the separation rests on the shallow locale tell (`src/detection/style_locale_control.py`). Removing every British and American spelling variant and every contraction from the text changes 1,251 of the 1,280 essays, but the function-words-only accuracy does not fall. It goes from 0.995 to 1.00, and the full TF-IDF model stays at 1.00. The locale and formality tells are real, then, but the separation does not rest on them; the distributional style signal is there with or without them. I keep the locale point as a caveat and a planned fix, not as the explanation for the score.

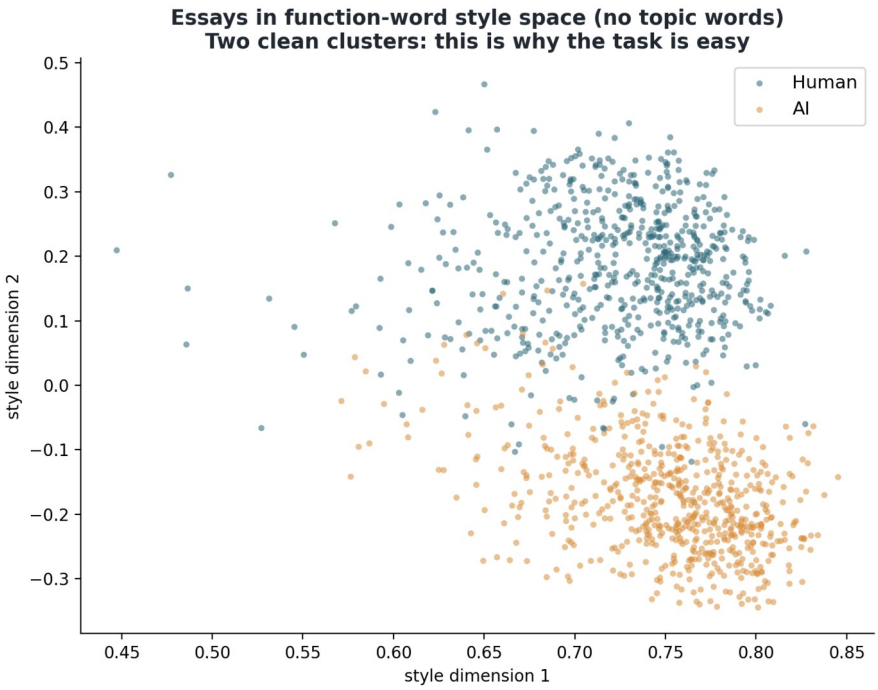


Figure 3.4: Every essay placed by its function-word style alone, with all topic words removed. Human and AI fall into two separate clouds, so a single generator in one domain is easy to separate.

The closest call came out reassuring. The human essay the model rates most AI-like scores 0.43 on the AI scale, so it is still correctly called human. The borderline cases are a small mix of native and non-native writers, not a pile of non-native essays. A cluster of non-native writing at the boundary was the bias I had been watching for, and it did not appear.

3.8 Where this chapter's threads are picked up

The audit also gives the explainability layer its starting point. The cleaned linear model can already point at the words that drove a decision, a first concrete version of the explainable, defensible evidence the later chapters develop. The attribution methods (Integrated Gradients, SHAP and attention) are built and faithfulness-tested in Chapter 5, the M4 multi-generator and cross-domain tests are in Chapter 6, and the stylometric features are fused with the transformer into the hybrid detector in Section 6.7.

Chapter 4: Implementation

4.1 Environment and reproducibility

The whole project runs on my own laptop. After Meeting 3 it became clear that no ATU HPC or cloud would be available, so every part has to fit a single RTX 4060 with 8 GB of VRAM, or fall back to the CPU. Most of the choices below follow from that limit, including the use of base-sized models instead of large ones.

The stack is Python on Windows, with PyTorch and the Hugging Face libraries for the transformer work, spaCy for the linguistic features, and scikit-learn for the simpler baselines and the audit probes. To keep runs repeatable I set a single random seed (42) everywhere that randomness enters: the sampling, the train and test split, and the model training. Dependency versions are pinned. The work is committed to a local git repository in small steps with clear messages, so any result can be traced back to the exact code that produced it. Long jobs (essay generation, model training) are launched as detached background processes instead of being tied to an open terminal session, so they keep running if the session that started them closes. This sounds like a small detail, but it cost me two failed generation runs before I worked it out, and it is now the standard way I start anything that takes more than a few minutes.

4.2 Data acquisition and cleaning

The human side of the corpus comes from the British Academic Written English (BAWE) collection (Alsop and Nesi, 2009), which I downloaded from the Oxford Text Archive. A first script (`src/data/explore_bawe.py`) reads the holdings spreadsheet, prints a summary, and cross-checks the recorded word counts against a plain count of the text files so I know the metadata is trustworthy. A second script (`src/data/clean_bawe.py`) produces a cleaned metadata table, dropping a row that was labelled twice. Every later step samples from the cleaned table.

4.3 Sampling

The sample is drawn by `src/data/build_sample.py`. I stratify evenly across the four broad disciplinary groups rather than by named discipline, and I balance native against non-native English writers so that I can later test whether the detector is unfair to non-native

writers. A per-student cap stops any one person's style from dominating. The train, validation and test split is made at the level of the student, not the essay, so the same writer never appears on both sides of the split. The result is 640 human essays with a versioned manifest (`bawe_human_sample_manifest.csv`) that records, for each essay, its group, first-language status, and split. The reasoning behind the sizes is written up in `dissertation/sample_design.md`.

4.4 Generating the matched AI essays

`src/generation/generate_ai_essays.py` builds the AI half of the corpus. For each human essay it generates one AI essay on the same topic and at the same target length, using Llama 3.1 8B running locally through Ollama. Matching on length and topic was the part I took most care over. An AI half that ran systematically longer or shorter, or that drifted onto different subjects, would let the detector separate the classes for the wrong reason, and the whole corpus would be invalid.

The matching works through the prompt and a length loop. The prompt is anchored on keywords pulled from the human source text, not just the essay title, after an early test with a title alone produced an off-topic essay. The generator also runs a short continuation loop. If the first draft falls short of the target length, it asks the model to continue, up to a few rounds, so the AI essay lands close to its human counterpart. The script is resumable, so a run that stops can be restarted and it skips what is already done, and it keeps the machine awake while it runs. Each essay goes to disk along with a metadata row recording the target and actual length, the number of rounds, and the time taken.

Generation of all 640 essays completed locally with no failures. A validation script (`src/generation/check_ai_corpus.py`) confirmed the match: the correlation between each AI essay and its human source length is about 0.98, the mean length ratio is about 1.05, and over 99 percent of AI essays are within 20 percent of their source length. A keyword spot-check confirmed the AI essays stayed on the same topics. The detector needs the two halves to differ only in the writing, and on these checks they do.

4.5 Building the labelled corpus

`src/detection/build_detection_corpus.py` pairs each human essay (label 0) with its matched AI essay (label 1), carries the split and metadata across from the manifest, and

writes a single table of 1,280 rows. The same script has a cleaning mode (described in Section 4.7) that strips markup before writing, so the raw and cleaned corpora can be compared directly.

4.6 The detector

Detector training lives in `src/detection/train_detector.py`, which uses the Hugging Face Trainer to fine-tune a transformer to classify human against AI. DeBERTa-v3-base is the primary model and RoBERTa-base the comparison. To fit the 8 GB card I keep the per-step batch small and use gradient accumulation to reach a sensible effective batch, with mixed precision on the GPU. The script reports accuracy, precision, recall, F1 and the confusion matrix. It also breaks out the false-positive rate for native and non-native writers separately, and the fairness analysis later builds on that breakdown. The stylometric feature extractor (`src/detection/stylometric.py`) computes the linguistic features (sentence-length variation, vocabulary richness, part-of-speech mix, and so on). Those features are fused with the transformer into the hybrid detector in Section 6.7 (`src/detection/hybrid_fusion.py`).

4.7 The audit and the cleaning step

The first detector scored a perfect 100 percent, which I did not trust. Before accepting it I ran an audit, implemented in three small scripts, and I count those scripts as part of the contribution.

`src/detection/audit_detector.py` runs the diagnostic battery: it confirms there is no student or duplicate-text leakage across the splits, measures how much of the score a markup-only rule can reach on its own, and trains simple interpretable baselines on raw and cleaned text, including a function-words-only model that removes all topic information. `src/detection/text_normalize.py` is the cleaning function the audit relies on. It strips the BAWE export tags from the human text and the markdown from the AI text, and flattens the layout, so that whatever difference remains comes from the writing and not the formatting. `src/detection/why_high.py` then explains why the cleaned score stays high. It measures locale spelling, contraction use, and how tightly the AI essays cluster in style space, and it draws the two classes in a two-dimensional style projection.

The finding, covered in Chapter 3, was that the original human text carried structural tags that no AI essay had, and that this shortcut accounted for a large part of the perfect score. The cleaning step removes the shortcut. The detector is then retrained on the cleaned corpus, and the retrained score is the headline figure I report.

4.8 Engineering notes and lessons

A couple of practical points from this phase belong in the methodology. One is launching long jobs as detached processes, which is what finally made the overnight generation reliable. The other is the markup artefact itself. It shows why a strong result has to be stress-tested before it is believed, and because the audit is scripted, anyone can rerun it and see the same thing. Both points feed into how I will build and check the remaining components.

Chapter 5: Explaining the detector's decisions

5.1 What this chapter does

A detector score on its own does not give a lecturer much to work with. If the system flags an essay, the lecturer needs to see why it was flagged, and needs some assurance that the stated reason is real. This chapter builds the first version of the explainability layer. It shows which parts of an essay drove the decision, and it then tests whether that explanation is faithful, which means whether the words it points to are actually the words the model used.

5.2 Token attributions with Integrated Gradients

For the word-level view I used Integrated Gradients, a standard attribution method, through the Captum library (`src/explainability/integrated_gradients.py`). It works on the detector's word embeddings and measures how much each token pushes the decision toward the AI class, compared to a neutral baseline where the content is removed. I attribute everything to the AI class so the sign is consistent. A positive score pushes toward AI, a negative score pushes toward human.

Figure 5.1 shows this for a matched pair, the AI and human versions of the same essay. The detector is confident and in the right direction for both (the AI essay scores 1.00 on the AI scale, the human essay 0.01). The heaviest tokens are punctuation and very common words and word-fragments: full stops, commas, a dash, words like "have", "be", "can", "similarly". Topic words barely feature. This matches what the audit found. The model reads style and rhythm, the small machinery of how a sentence is put together, more than it reads what the essay is about.

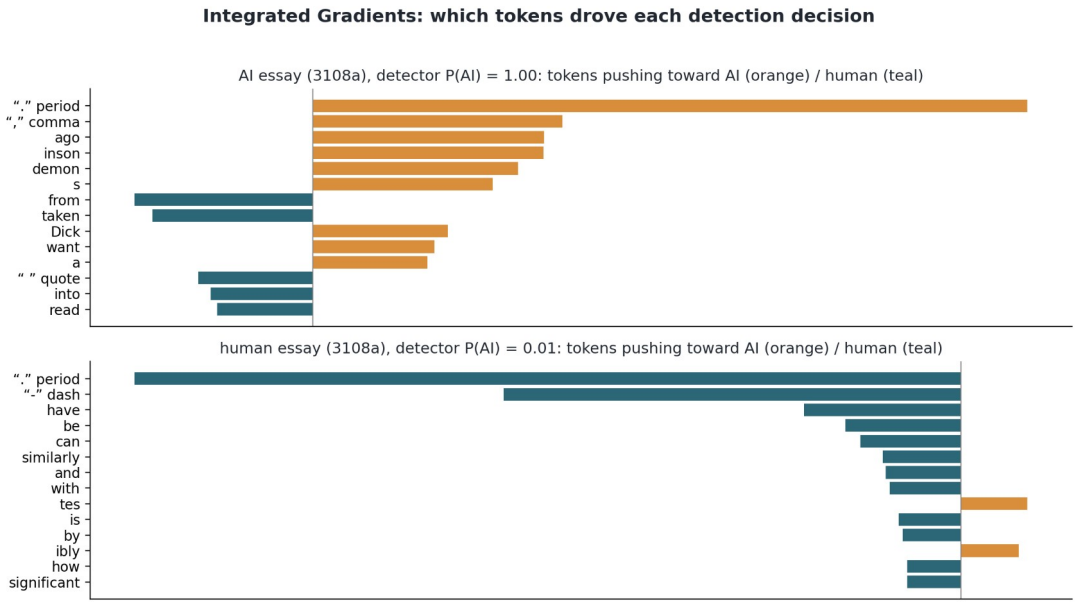


Figure 5.1: Integrated Gradients token attributions for a matched AI and human essay. Orange pushes toward AI, teal toward human. The strongest tokens are punctuation and common words; topic words carry little weight, so even at the word level the detector is reading style.

5.3 Testing the highlights for faithfulness

Highlights can look convincing without reflecting what the model actually used, so I tested them by ablation in the style of the ERASER benchmark (DeYoung et al., 2020), in the same script. For a sample of test essays I ranked the tokens by their attribution and ran two checks. Comprehensiveness asks whether removing the top tokens lowers the detector's confidence, and whether it lowers it more than removing the same number of random tokens. Sufficiency asks whether keeping only the top tokens and hiding the rest is enough to hold the confidence up.

Figure 5.2 shows the comprehensiveness result as more and more tokens are removed. Removing the top-ranked tokens does hurt the detector more than removing random ones, but only by a little, and only once a fair number are removed. The comprehensiveness sweep is the part of the test I rely on. I also ran the sufficiency check (keep only the top tokens, hide the rest), but with at most a few dozen of 256 tokens kept the model sees almost nothing and sits at a coin-flip for every k. That flat line is an artefact of feeding the model near-empty input, so I do not treat it as a finding. The comprehensiveness result on its own shows that no small set of words carries the decision.

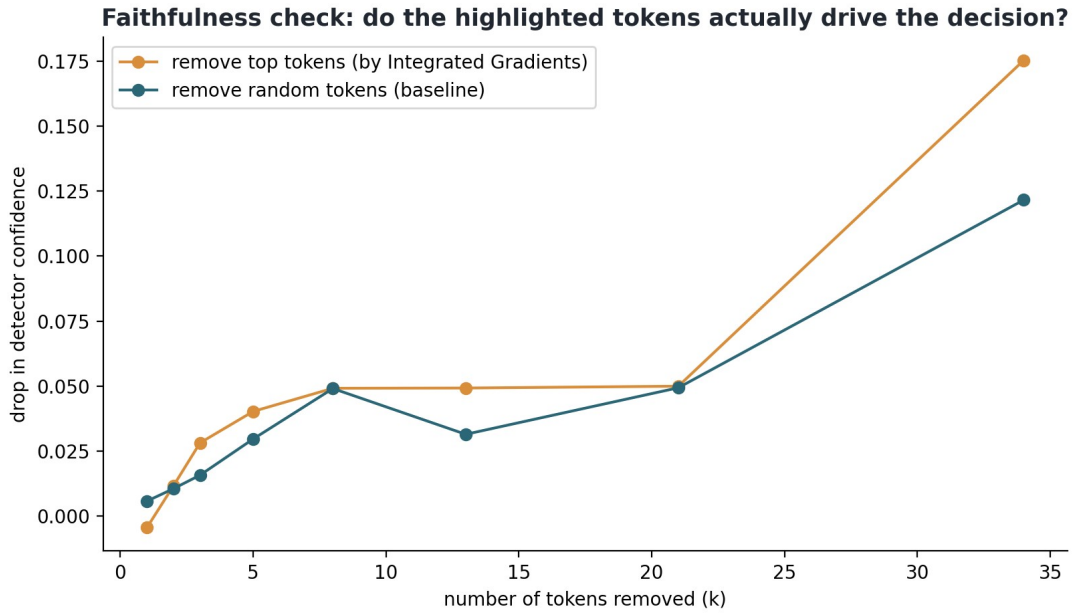


Figure 5.2: Faithfulness by ablation. Removing the top attributed tokens (orange) lowers detector confidence only slightly more than removing random tokens (teal). Keeping only the top tokens collapses the prediction to chance. The signal is spread across the whole essay.

5.4 The signal is diffuse

This is a negative result, but a useful one. A few words cannot explain the decision because the difference between human and AI writing here is spread across the whole essay, in the ordinary words and the punctuation. There is no handful of give-away terms to point at. The audit found the same thing from the other direction: a model using only function words still separated the classes at 99.5 percent, and the essays form two clean clouds in pure style space. A diffuse signal is hard to summarise with a word-level highlight, so for this detector a per-word heatmap is a weak explanation on its own.

The feature-level explanation is the one that holds up for this detector. The interpretable linear view from the audit (Chapter 3) names the style markers that separate the classes, the large-model register on one side and the blunter connectives on the other, and the function-words-only result backs that view up. For the lecturer-facing output I therefore lean on the style and feature picture, together with the source-grounded verification questions that come later in the pipeline. A single highlighted sentence would be a weaker basis.

5.5 Stylometric features and SHAP

To make the feature-level explanation concrete I built a detector from hand-crafted style features alone, with no transformer: sentence-length variation and burstiness, vocabulary

richness, word length, punctuation, and the part-of-speech mix (`src/explainability/shap_stylometric.py`). Trained on the same student-level splits, this transparent model reaches an F1 of 0.985 on the test set (95% bootstrap confidence interval about [0.97, 1.00], and identical, 0.985, across five training seeds, so it is stable). The comparison with the transformer's 0.99 is not quite like for like, because the feature model reads the whole essay while DeBERTa reads only the first 512 tokens. That helps the feature model, so I describe the two as "competitive" rather than "equal". The false-positive rate of 0.02 for native and non-native writers also needs care. It works out at one essay in fifty each, too small a base for any fairness claim; the cross-domain result in Chapter 6 is the real fairness evidence. Two of the features (type-token ratio and the rare-word ratio) are length-sensitive, but since the essay lengths are matched across the classes this carries little class signal, and the length-robust vocabulary measure behaves the same way. Perplexity, one of the strongest stylometric signals in the literature, needs a GPU pass and joins the feature set when the hybrid is assembled in Section 6.7, where it proves to be the strongest single feature. Even with these caveats, a handful of interpretable features captures the signal almost as well as a large model does.

Because the model is built from named features, I can explain it faithfully with SHAP, which attributes each decision to the features that drove it. Figure 5.3 shows the picture across the test set. Longer words and a denser use of auxiliary verbs push a text toward AI. More sentence-length variation, richer vocabulary, and more rare one-off words push it toward human. This is the kind of explanation the project needs. A lecturer can be told that a piece was flagged because its sentences are unusually uniform, its words longer, and its vocabulary less varied than a typical student's, and the account is faithful because the model literally uses those features. A black-box percentage gives the lecturer none of that.

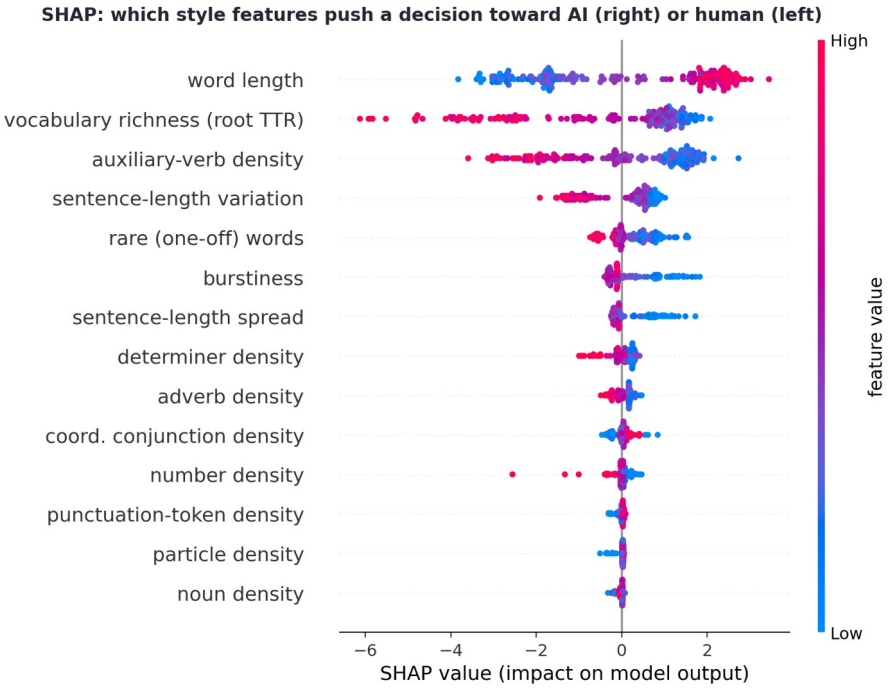


Figure 5.3: SHAP on the stylometric detector. Each dot is an essay; position shows how much a feature pushed the decision toward AI (right) or human (left), and colour shows the feature value. Longer words push toward AI; richer vocabulary and more varied sentence length push toward human.

This stylometric model is also the non-transformer half of the planned hybrid detector, so this step serves both the explainability layer and the detector itself.

5.6 Attention visualisation on the same test

Attention visualisation is the third of the three methods named for the explainability component (`src/explainability/attention_viz.py`). I use the standard view: how strongly the classifier position attends to each token in the final layer, averaged over heads. This is the view often presented as "what the model looked at". On the matched pair from Section 5.2 it produces a plausible-looking picture (Figure 5.4). In the AI essay the most-attended token is an em dash, followed by content words like "extract" and "poetry". In the human essay the attention mass sits on ordinary function words like "the", "as" and "is". The em dash detail also fits the stylometric results, since dash-heavy punctuation is one of the register markers the feature model picks up.

A plausible-looking picture can still be unfaithful, and attention weights have a reputation for being poor guides to what actually drives a decision. I therefore ran the same ablation protocol as in Section 5.3, keeping the test sample, the seed, and the k-sweep unchanged, so all three methods can be compared on one measure. Attention lands almost where Integrated

Gradients does (Figure 5.5). Removing its top 34 tokens drops confidence by 0.17 against 0.12 for random tokens, a ratio of 1.43 to IG's 1.44. Keeping only its top tokens leaves the detector near a coin flip (sufficiency around 0.53), just as it does for IG. At small k attention is even marginally ahead of IG, but neither method's top tokens come anywhere near sufficient to carry the decision.

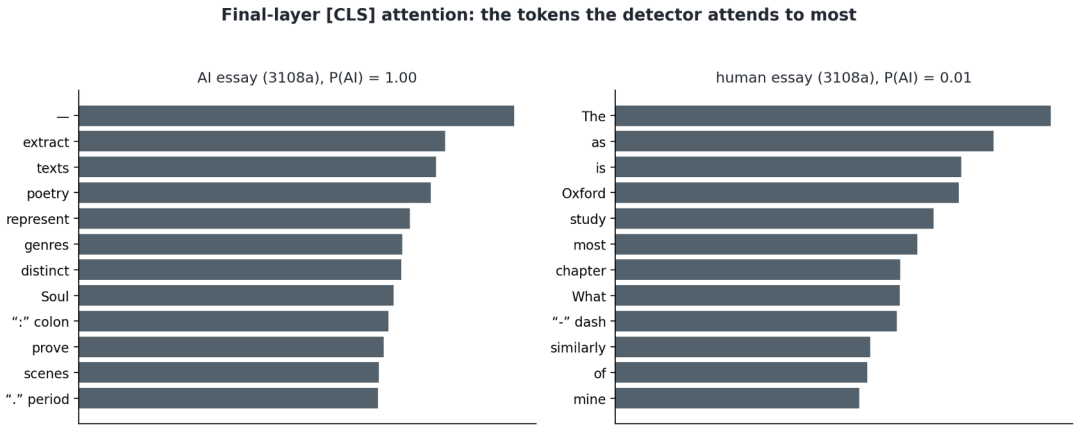


Figure 5.4: Final-layer attention from the classification position for the same matched pair as Figure 5.1. The picture looks interpretable, and in the AI essay the single most-attended token is an em dash, but whether it can be trusted depends on the faithfulness test.

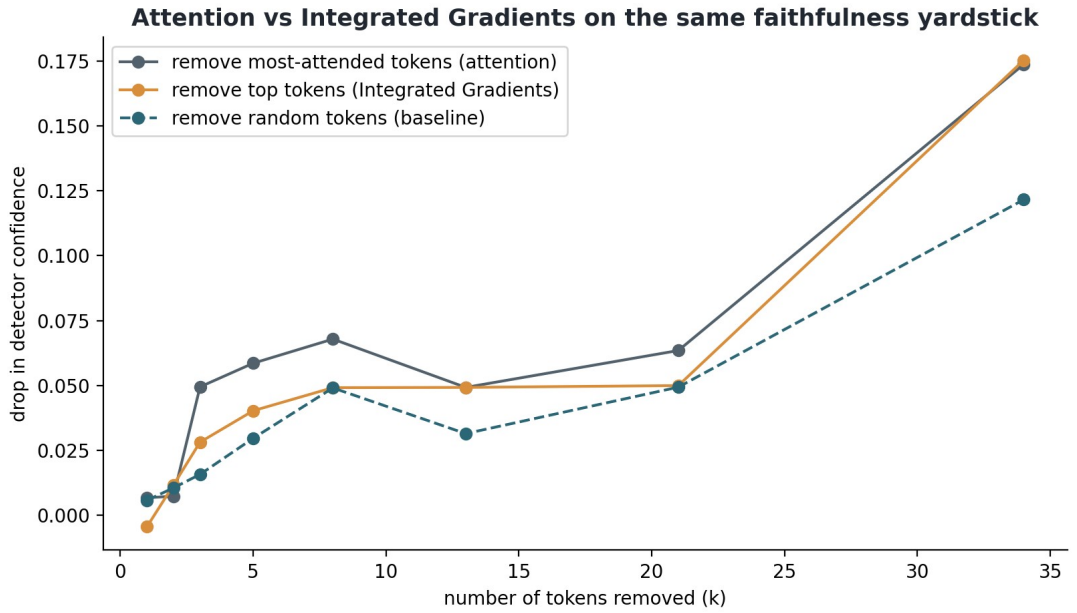


Figure 5.5: Attention and Integrated Gradients on the identical faithfulness protocol. The two token-level methods are nearly indistinguishable, and both sit only modestly above the random-removal baseline.

Measuring the third method leaves the conclusion where it was. Attention and Integrated Gradients agree with each other, and both are weakly faithful because the style signal is spread across many ordinary tokens. SHAP over the named stylometric features passes its ablation test and stays as the explanation shown to lecturers. Having the third method

measured still adds something. Two independent token-level techniques fail the same test in the same way, so the weakness belongs to token-level explanation on this task rather than to any one attribution method.

5.7 A lecturer-facing explanation card

The SHAP beeswarm of Figure 5.3 is the faithful explanation, but it is a researcher's picture. It takes training to read, and the audience for this system is a lecturer with none. My supervisor's feedback on the mid-project presentation made the same point, that the explanations convince the person who built them and not yet the person who must use them. So I added a second rendering aimed at that reader (`src/explainability/explain_submission.py`), and it is now part of every generated guide.

For one submission, the card takes the same style model the hybrid detector uses, computes which habits moved this particular essay's score, and states each one as a plain sentence with its number set against what is typical for real student essays in the corpus: "the sentences are unusually uniform in length, variation 7.5 against a typical 11.4", "a language model finds this text unusually easy to predict, surprise score 18 against a typical 30". Each sentence ends with the direction it pushed, and a small bar chart shows the same five habits with arrows toward AI or human. The wording is generated from the numbers themselves (higher or lower than the human median), so a sentence cannot contradict its own figures. The card also explains the very model that produced the score rather than a simplified stand-in, so the faithfulness shown in Section 5.5 carries over without needing a new argument.

The explainability work is not finished here. The card has not yet been tested on actual lecturers, its reference band is one corpus's idea of "typical", and the transformer half of the hybrid still contributes to the score without a per-essay account of its own. Those items are named in the plan. From this point on, any explanation the system shows has to pass two checks, the ablation test for faithfulness and a plain-reader test for whether it is understood.

5.8 Next steps for explainability

The stylometric model and the transformer are fused into the hybrid detector in Section 6.7, with perplexity added to the feature set. The token attributions stay in as a supporting visual, useful for showing a lecturer roughly where the model looked, always paired with the caveat

from the faithfulness test and with the SHAP feature view as the primary explanation. The faithfulness check itself carries forward as a standard step, so any explanation the system shows is one I have tested first.

Chapter 6: Robustness, transfer to unseen generators and domains

6.1 The question

The in-domain detector scores about 0.99. That score comes from an easy setting, one generator (Llama) and one kind of text (student essays). This chapter asks what happens when neither holds. I took the trained detector, with no retraining or adaptation, and ran it on the M4 benchmark (SemEval-2024 Task 8), which has many generators and many domains the detector never saw. I split the work into two tests so the easy half does not get labelled "robust" by accident.

A note on sources, because the two tests draw on different data. Test A uses the OUTFOX essay set that ships inside the M4 release, human essays against six generators. Test B uses the M4 / SemEval-2024 Task 8 monolingual data across five web and academic domains. I chose two different corpora deliberately. Test A isolates a change of generator while holding the text type fixed (essays). Test B changes the domain. Since they have different generator sets, I name them separately instead of calling both "M4".

6.2 Test A: transfer to unseen generators (still essays)

The first test uses the OUTFOX essay set, where humans and six different models (GPT-4, ChatGPT, Cohere, BLOOMz, Dolly, and davinci) all wrote essays. The kind of text stays the same as training; only the generator changes. The detector held up well. F1 was 0.97 (95% confidence interval [0.97, 0.98], on about 4,800 essays so the interval is tight), it caught every generator at between 96 and 100 percent (Figure 6.1), and the human false-positive rate was about 5 percent. The result is useful and it surprised me a little. A detector trained only on Llama still recognises text from quite different models. Current large models share a lot of the same smooth, uniform style, so the AI fingerprint largely carries across generators, at least on essays.

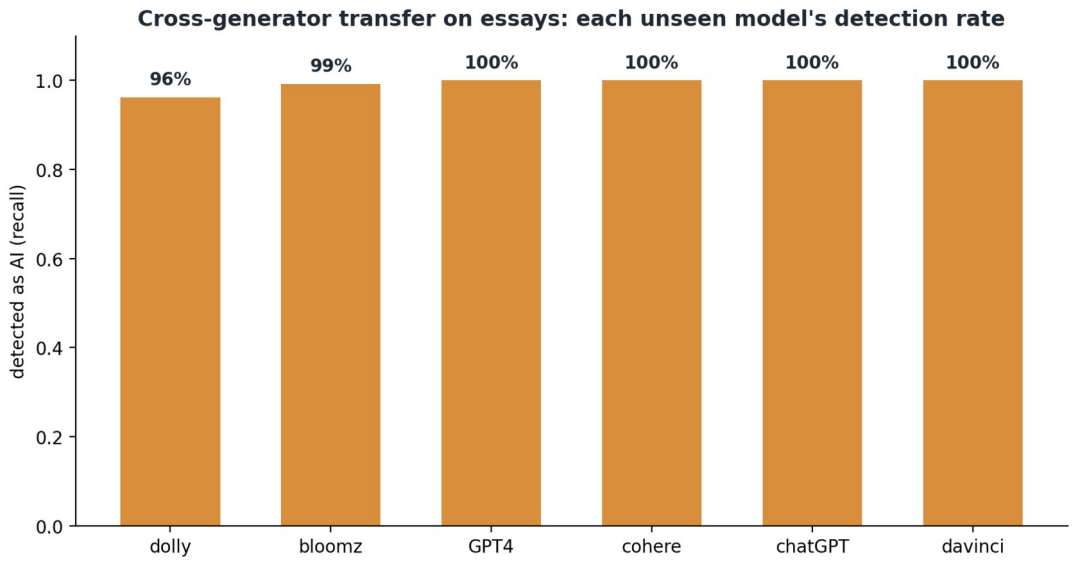


Figure 6.1: Transfer to unseen generators on essays. A detector trained only on Llama still flags GPT-4, ChatGPT, Cohere, BLOOMz, Dolly and davinci at 96 to 100 percent.

6.3 Test B: transfer to unseen domains

The second test is the hard one, on the M4 monolingual data. Here the human text is not essays at all. It is Reddit posts, WikiHow articles, arXiv abstracts, Wikipedia, and peer-review text, set against four generators. The overall F1 falls to 0.79 (95% confidence interval [0.77, 0.80], tight because the sample is large). The generator set here differs from Test A, so the headline drop mixes a change of domain with a change of generators. For that reason I treat the human side of the result as the stronger evidence, because a false-positive rate measured on human text cannot be a generator artefact. The detector still catches the machine text well in every domain (86 to 98 percent). The failures are on the human side. The detector wrongly flags genuine human writing as AI at high rates on the more formal domains: 79 percent on arXiv abstracts, about 40 percent on Wikipedia and WikiHow, 30 percent on peer-review, and 23 percent on Reddit (Figure 6.3).

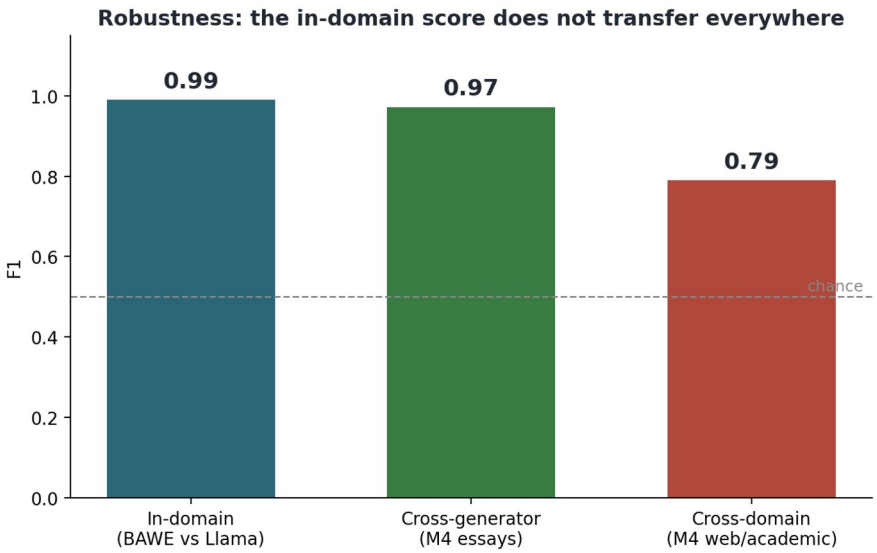


Figure 6.2: The in-domain score does not transfer everywhere. F1 stays high across generators on essays but falls on out-of-domain human and AI text.

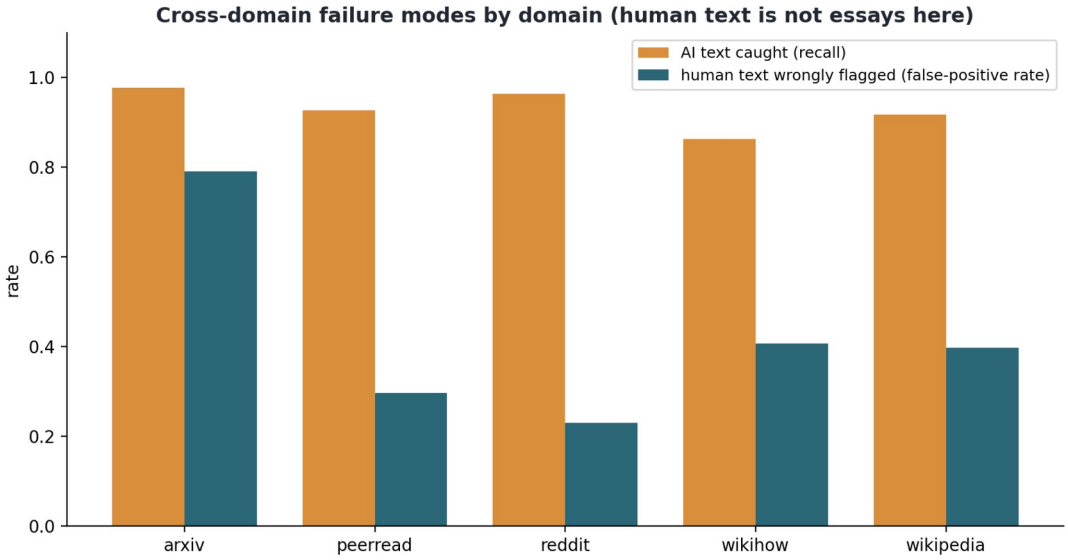


Figure 6.3: Cross-domain failure modes. The detector still catches AI text (orange) but wrongly flags formal human text as AI (teal), worst on arXiv abstracts.

6.4 What this means

Taken together, the two tests show a detector that transfers across AI models and fails across domains, and the failure has a pattern. The model learned that "human" looks like a student essay. When it meets human writing that is more formal or more technical, an arXiv abstract for example, it calls it AI. So the out-of-domain failure is a false-accusation failure, the harm the project set out to avoid.

This connects to the fairness concern from the start of the project. A detector trained on one narrow idea of human writing will misjudge writers whose style sits outside that idea. The

arXiv result shows this most clearly, since formal, dense, careful human prose reads as machine to this model. The same mechanism makes these tools risky for non-native writers, whose style also differs from the training norm. It is also the strongest reason the pipeline does not stop at a score. A number that is confidently wrong on a whole domain is dangerous on its own. The verification questions give a lecturer something to check before acting on a flag.

6.5 Caveats and next steps

This is a zero-shot test with no adaptation, so it is a lower bound on what is achievable. A detector trained on diverse human text and several generators would almost certainly do better across domains, and that is a clear next experiment (train on a slice of M4 and re-test). The false positives trace back to the in-domain "human" class being essays, so the fix is more varied human data and per-domain calibration. The remaining robustness gaps to test are paraphrased and "humanised" AI text, and documents that mix human and AI writing. As the project stands, the detector is good at spotting AI essays from many models. It is not safe to point at human writing from a domain it was not trained on.

6.6 The natural-distribution control

One design worry from Meeting 2 with my supervisor stayed open until now. The detection corpus is deliberately balanced, with equal essays per disciplinary-group-by-native cell, but real submissions look nothing like that. In the full BAWE corpus, native Arts and Humanities writers supply about 21 percent of essays and non-native AH writers about 4 percent. If the detector's strong results depended on the artificial structure, the whole evaluation would be over-fitted to a corpus shape that never occurs in practice. To check, I trained two detectors in parallel from the same cleaned corpus with identical hyperparameters, seeds and size (247 human essays each, plus their matched AI twins). They differ only in writer mix: one balanced (31 per cell), one matching full BAWE's natural skew (53 native AH essays down to 10 non-native AH). Both are evaluated on the same untouched test split, per cell, reweighted to the natural proportions, and with the fairness read (`src/evaluation/balanced_vs_natural.py`, `outputs/balanced_vs_natural.json`).

The balancing turns out not to matter (Figure 6.4). The two models score identically, F1 1.000 overall, in every cell, and under natural reweighting. They make the same prediction on every one of the 200 test essays (zero disagreements, McNemar p = 1.0), with a zero false-positive rate on human essays in both native-language groups. Given Chapter 3 this is unsurprising. The cleaned corpus is separable on function words alone, so 494 training essays saturate the test set regardless of how their writers are distributed. On the reassuring side, the balanced design did not manufacture the result, because a realistically skewed training mix reproduces it. That answers the Meeting 2 worry. The caveat is that at this level of separability the comparison has no resolution. Whether balancing helps or hurts can only be measured where the task is hard, such as the cross-domain settings of this chapter, and a future version of this control belongs there.

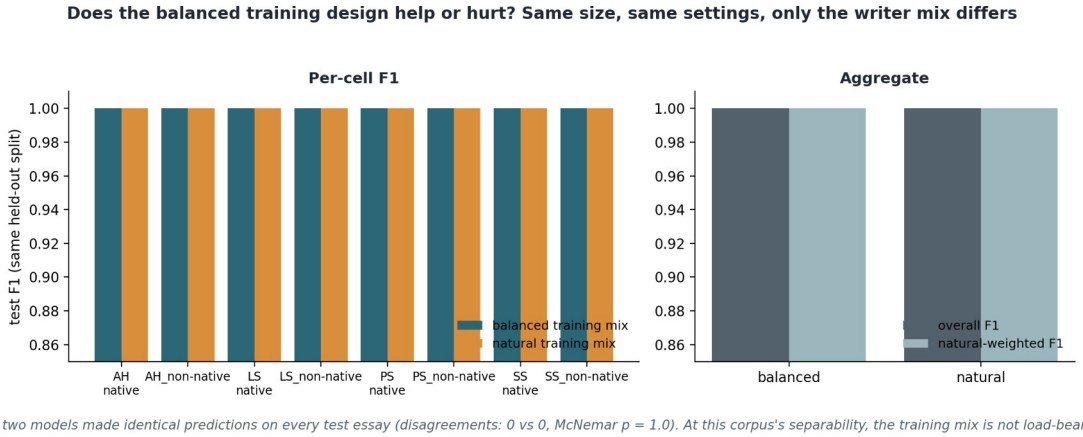


Figure 6.4: The training-distribution control. Two same-size detectors, one trained on the balanced writer mix and one on BAWE's natural skew, evaluated per cell and in aggregate on the same held-out split. The two are indistinguishable everywhere, so the balanced design does not explain why the corpus is separable.

6.7 The hybrid detector

The first component in the scope is a hybrid, the transformer combined with the stylometric features, including the GPT-2 perplexity signal (Radford et al., 2019) that Chapter 5 deferred because it needs a GPU pass. Both halves have existed for some time at 0.990 and 0.985 in-domain, so assembling the hybrid was never going to move the headline number. After the results above, the question I cared about was whether fusion changes the detector's behaviour where it fails. The assembly is simple. Perplexity joins the feature set, the feature model is retrained, and a logistic regression fuses the two models' probabilities. The regression is fitted on the validation split so the test set stays untouched, then applied zero-

shot out of domain (`src/detection/hybrid_fusion.py`, `outputs/hybrid_fusion.json`).

In-domain, everything sits at the ceiling. Adding perplexity lifts the feature model from 0.985 to 1.000 on the 200-essay test split, and the hybrid scores the same. Two essays' worth of movement separates all the arms, so these differences should not be over-read. One detail is worth keeping. Perplexity enters the feature model as its strongest single feature, first of twenty-five by mean absolute SHAP. That matches the literature's regard for it, and it completes the feature set the scope specified.

Out of domain the fusion matters (Figure 6.5). On the same cross-domain sample as Section 6.3, the three arms have almost identical F1 (transformer 0.790, hybrid 0.791), but they make completely different errors. The transformer over-flags humans, with recall 0.93 at precision 0.69, the false-accusation profile from Section 6.3. The style-plus-perplexity model errs the other way, precision 0.82 at recall 0.72. The fusion balances the two (precision 0.80, recall 0.78) and lifts accuracy from 0.753 to 0.794. The larger effect is on the human false-positive rate, which falls in every domain: from 0.79 to 0.61 on arXiv abstracts, 0.30 to 0.09 on peer reviews, 0.23 to 0.05 on reddit, 0.41 to 0.10 on wikihow and 0.40 to 0.12 on wikipedia. In four of the five domains false accusations drop by a factor of three to eight. arXiv remains the hard case, better but not fixed. The style features drive the improvement, because hand-crafted features generalise across registers while the transformer's learned representation of "human" stays anchored to student essays.

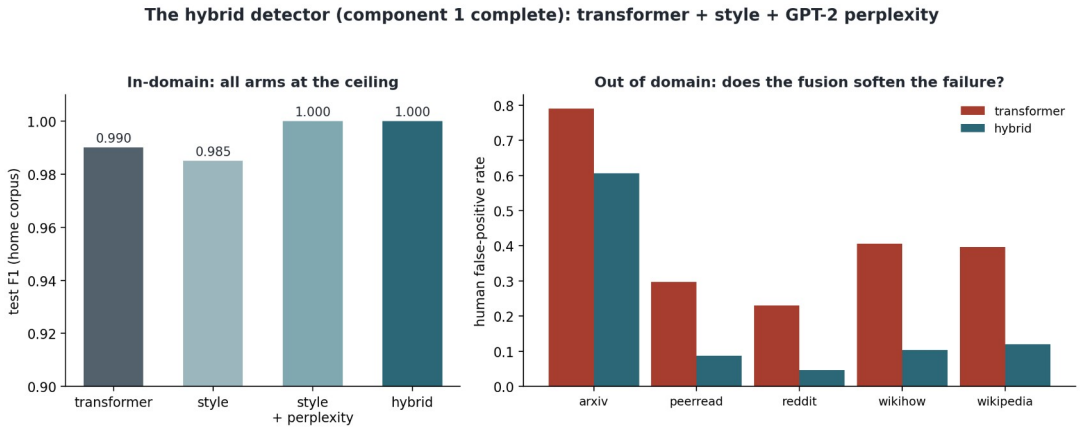


Figure 6.5: The hybrid detector. In-domain (left) every arm is at the ceiling and the differences are within noise. Out of domain (right) the fusion barely moves F1 but changes the error mix. The human false-positive rate, the failure described in Section 6.3, falls in every domain, by three to eight times in four of the five.

This connects back to the mitigation plan in Section 6.4. Fusion does not solve domain shift, and the headline F1 barely moves, so it is not a robustness cure. It redistributes the remaining errors away from falsely accusing human writers, the direction a deployed system must err in, and the cost is a feature extractor that runs anywhere. Component 1 of the scope is now complete as specified. The rest of the pipeline should call the hybrid rather than the bare transformer.

6.8 Measuring the abstain band

Sections 6.4 and 9.5 propose an abstain band as a deployment safeguard. When the hybrid's probability falls in a middle band, the system says "uncertain" instead of flagging. This section measures what that buys. On the same cross-domain sample as Section 6.7, keeping the per-text hybrid probabilities this time, I swept bands from none to 0.2-0.8 (`src/detection/abstain_band.py`, `outputs/abstain_band.json`).

Part of the answer matches the proposal (Figure 6.6). Accuracy among the texts the system still judges climbs steadily, from 0.79 with no band to 0.88 when the widest band declines 28.5 percent of texts. Abstention does concentrate the detector's verdicts on cases it gets right. But the human false-positive rate among judged texts barely moves. It sits near 0.19 across the whole sweep. The false accusations that survive the hybrid sit far from the threshold. They are confident errors, mostly the dense arXiv abstracts that the detector scores as machine-like with conviction, so an uncertainty band never sees them. I had expected the band to cut false accusations, and the sweep shows it does not.

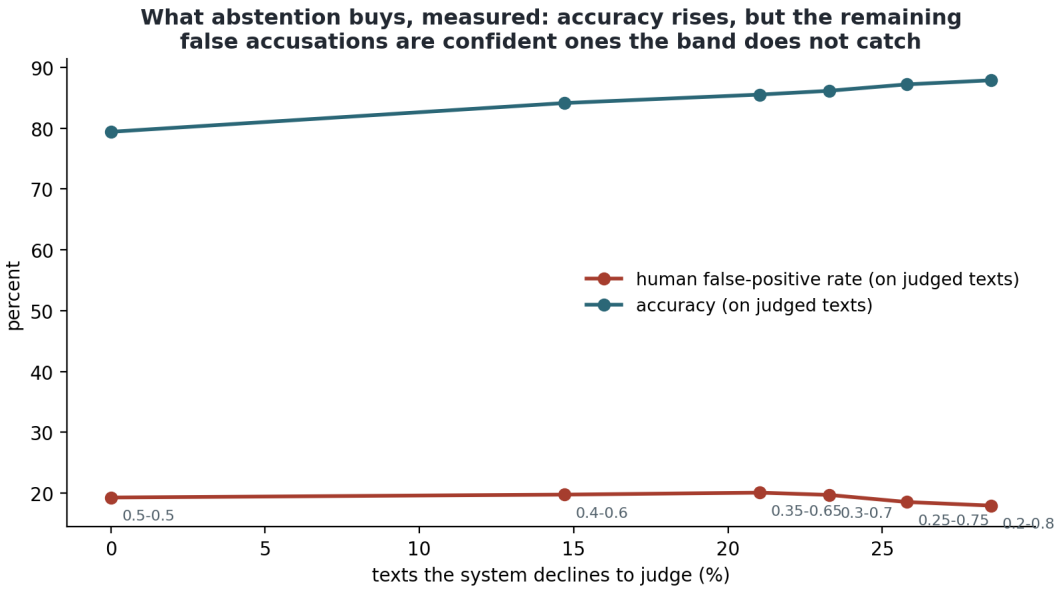


Figure 6.6: The abstain band swept from none to 0.2-0.8 on the cross-domain sample. Accuracy on the judged texts rises steadily, but the human false-positive rate barely moves, because the surviving false accusations are confident errors that fall outside any uncertainty band.

This updates the mitigation plan. Abstention is worth deploying, since a quarter of verdicts withheld buys nine points of accuracy. It does not fix the fairness problem. The confidently wrong flags need per-domain calibration or domain detection before the score can be trusted at all, and until then the question stage remains the backstop for these cases. The band's benefit and its limit are both now measured and on the record.

6.9 Unseen commercial generators on the home corpus

One gap remained in the robustness picture. The cross-generator evidence (Section 6.2) lives on an external benchmark with its own corpus design, so this section repeats the test on the home corpus. For forty test-split human essays, I generated matched AI counterparts with two commercial models the detector never saw, Gemini 2.5 Flash and GPT-4o-mini, using the same recipe as the original corpus, the same system prompt and the same title-plus-keywords-plus-target-length user turn (`src/generation/multigen_test_slice.py`). Gemini completed nineteen of the forty before its free tier ran dry, and GPT completed all forty. Because GPT-4o-mini undershoots its target length on many essays (mean ratio 0.45), the length ratios are recorded per essay, and results are reported both for all essays and for the reasonably matched subset. The two agree (`src/detection/score_multigen.py`, `outputs/multigen_detection.json`).

The transformer of record generalises perfectly here. It flags 100 percent of both unseen generators' essays and none of the forty human sources. The hybrid behaves differently. It catches all of GPT-4o-mini but only 68 percent of Gemini's essays. The style half, the same component that cut false accusations of humans by three to eight times in Section 6.7, reads a third of Gemini's essays as human-like enough to pull the fused score under the threshold. This is the expected cost of a fusion built to err toward not accusing, now measured on its other side. A detector tuned to protect unusual human writing will extend some of that protection to a generator whose style drifts toward human. For deployment the conclusion matches Section 6.8's. The fusion sets the right default, and the question stage exists because no threshold can be both safe for humans and airtight against every generator at once.

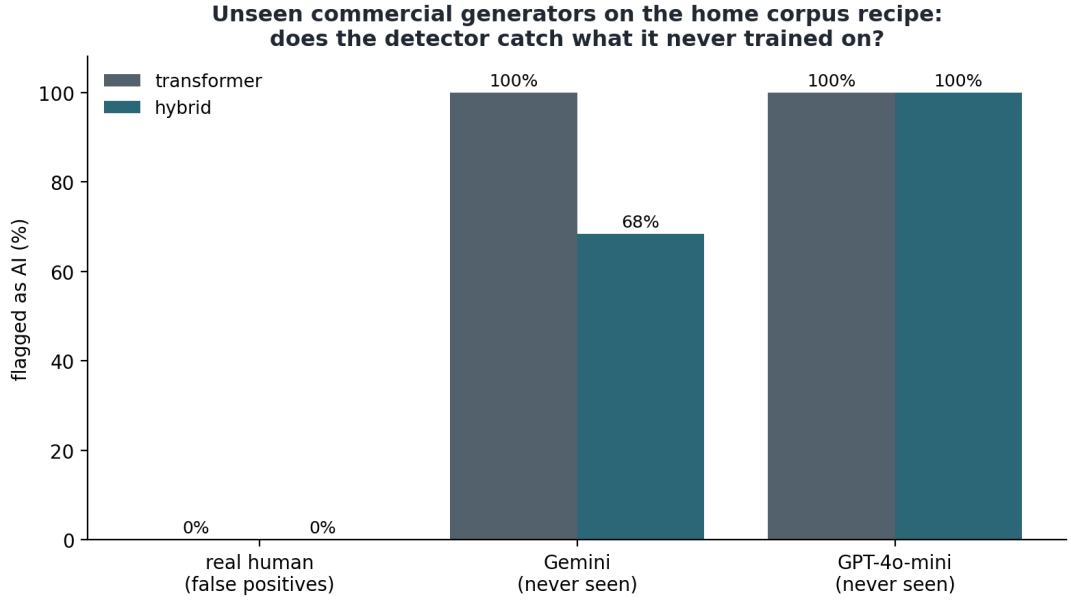


Figure 6.7: Unseen commercial generators produced with the home corpus recipe. The transformer flags every AI essay from both new generators and no human sources; the hybrid trades 32 percent of Gemini recall for the false-accusation protection measured in Section 6.7.

Chapter 7: From a flag to verification questions

7.1 The step after detection

Detection on its own only produces a flag, and the earlier chapters showed how fragile a flag can be out of domain. The real contribution of the project comes after the flag. A suspected submission is turned into something a lecturer can act on fairly: a short set of questions, drawn from the student's own claims, that check whether the student understands what they handed in. A student who wrote and understood the work can answer them. A student who did not will struggle. This chapter builds that step. It starts with a thin slice that runs end to end, then replaces the slice's stand-ins one by one with trained components, and finishes with the fine-tuning of the local backend.

7.2 How the slice works

The pipeline takes a flagged essay and produces a Verification Interview Guide (`src/question_gen/generate_questions.py`). It runs in four steps.

First, it splits the essay into numbered sentences. Second, it extracts the student's main claims and records the sentence numbers each claim comes from. The design choice that matters here is provenance. The model is asked to cite sentence numbers instead of quoting text, and the source is then looked up from the real numbered sentences, so the model cannot invent a quote. Every claim points at exact lines in the submission. Third, for each claim it generates verification questions that are grounded in the source sentences and written so that someone who only read the claim cannot answer them well. Fourth, it tags each question with a Bloom's cognitive level (Anderson and Krathwohl, 2001).

Backends sit behind one interface. This first slice runs on the local open-source model (Llama 3.1 8B through Ollama), which is the basis for Backend B. The commercial Backend A plugs into the same interface, and the backend used is recorded with every guide, so the two can be compared directly. The comparison between the two backends is the core commercial-versus-local research question, and Chapter 8 carries it out.

7.3 An example

Run on a flagged essay that compares three literary texts, the system pulled out four claims and wrote grounded questions for each. Two of them give the flavour.

One claim was that the three texts use language differently (metaphor and symbolism in the poetry, dramatic intensity in the drama, subtle nuance in the prose), tied to two specific sentences in the submission. For it the system asked: "What specific features of Shakespeare's dialogue lead you to describe it as having dramatic intensity?" and "How does Austen's subtle and nuanced prose style allow her to explore complex social issues in a way that might be less effective with more overt language?" Answering either one takes actual reasoning about the texts. Repeating the claim back would not be enough.

Another claim was that the texts share themes of love and personal growth, tied to a single sentence. Here the system asked for an example from one of the texts that links personal growth to relationships. Producing such an example is only possible for a student who genuinely engaged with the texts. The full guide, with every claim, its exact source sentences, and the Bloom's levels, is saved as `outputs/verification_guides/3108a_ai.md`.

7.4 Stand-ins in the first slice

The slice above was deliberately a thin path through the remaining pipeline, with three declared stand-ins. They are worth recording here because the rest of this chapter replaces them one by one. The claim extraction was prompted rather than the trained argument miner planned in the scope; the trained miner arrives in Section 7.6. The Bloom's tag was a transparent keyword heuristic, replaced by the trained classifier in Section 7.5. And the questions themselves were not yet evaluated; the discrimination simulation that turns them into measured results is the subject of Chapter 8, where the commercial backend also joins the comparison. One decision was still open at this point, whether the local backend should be the full Llama 3 8B with QLoRA or a smaller model that fits the laptop. My supervisor and I settled it on the evidence of the fit probes, and Section 7.8 picks it up.

7.5 The Bloom's classifier

The keyword heuristic above was always a placeholder, and component 5 of the scope is a trained classifier. I fine-tuned BERT-base (Devlin et al., 2019) on the Bloom-labelled subset of EduQG (Hadifar et al., 2022). That subset holds 903 questions carrying a cognitive level, and only four levels occur (remember, understand, apply, analyse), in a heavily skewed distribution (660, 114, 110 and 19 examples respectively). Training used class weights against

the imbalance, stratified splits (632 train, 135 validation, 136 test), and six epochs. The keyword heuristic was scored on the same held-out test split as the baseline.

The trained model roughly doubles the heuristic: macro-F1 0.31 against 0.16, accuracy 0.57 against 0.26 (Figure 7.1). The per-class breakdown is less comfortable. BERT is reliable on the majority class (remember, F1 0.76) and moderate on understand (0.38), but it cannot learn apply (0.09) or analyse (0.0) from 110 and 19 examples. Coarsening the same predictions into the standard lower-order versus higher-order split does not rescue the minority side either. Higher-order F1 is 0.23 for BERT and 0.17 for the heuristic, and the heuristic's apparently high binary accuracy (0.86) comes from majority-class bias, since predicting lower-order for everything scores about 0.85 on this test set.

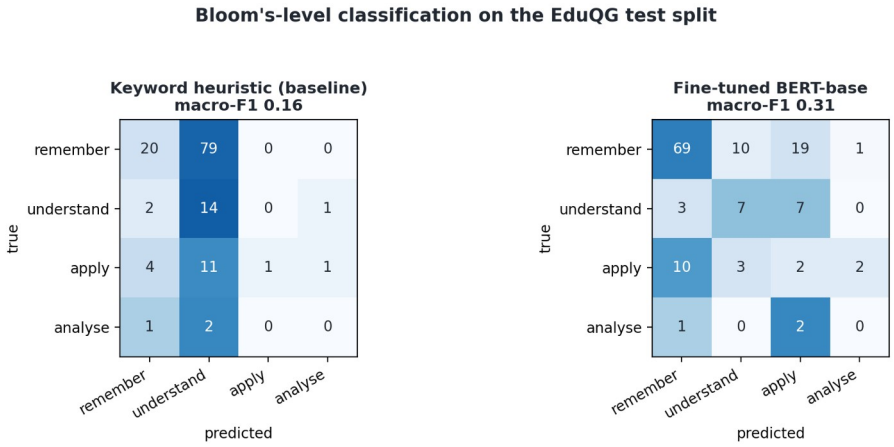


Figure 7.1: Bloom's-level classification on the EduQG test split. The trained BERT-base doubles the keyword heuristic on macro-F1 (0.31 vs 0.16), but neither model can learn the two smallest classes from 110 and 19 examples.

The component works and clearly beats the transparent baseline, so it replaces the heuristic in the pipeline. The bottleneck is the labelled data, not the model. In EduQG, 73 percent of the labels sit at the lowest level, so a classifier that has to recognise higher-order questions needs either more labelled examples of them or a coarser, better-populated label scheme. For the guide's quality-control purpose this means Bloom's tags on remember and understand questions can be trusted, while tags on the higher levels should be treated as suggestions until the label supply improves. Both the model and the full metrics are saved (`models/bloom_classifier/`, `outputs/bloom_classifier.json`).

7.6 The trained argument miner

The prompted claim extraction above was the declared stand-in for component 3, and the trained version now exists. I downloaded the Persuasive Essays 2.0 corpus from its official repository (402 essays, BRAT annotations for major claims, claims, and premises, with the corpus's own 322/80 train/test split) and fine-tuned DeBERTa-v3-base as a BIO token classifier over the three component types. Sequences are paragraphs. No annotated component crosses a paragraph boundary, and the longest paragraph fits inside 256 subwords, so nothing is truncated. Evaluation is strict span-level (exact boundary and exact type) with seqeval on the held-out test essays.

The result is a micro span-F1 of 0.63: premises 0.72, major claims 0.54, claims 0.44 (Figure 7.2). Strict matching is a hard yardstick, and the per-class order is the expected one, since claim boundaries are the classic ambiguity in this corpus. Dedicated argument-mining architectures with CRF decoding or joint relation modelling report higher figures (Pietron et al., 2024), so this is a working first version some way short of the state of the art, and I report it as such. For the pipeline it opened a design choice, settled with my supervisor and implemented in Section 7.9. The trained extractor finds spans in the student's own words. The prompted extractor produces readable claim paraphrases with sentence citations. The guide combines them, using the spans for provenance and the prompted paraphrases for phrasing. The model and metrics are saved (`models/claim_extractor/`, `outputs/claim_extractor.json`).

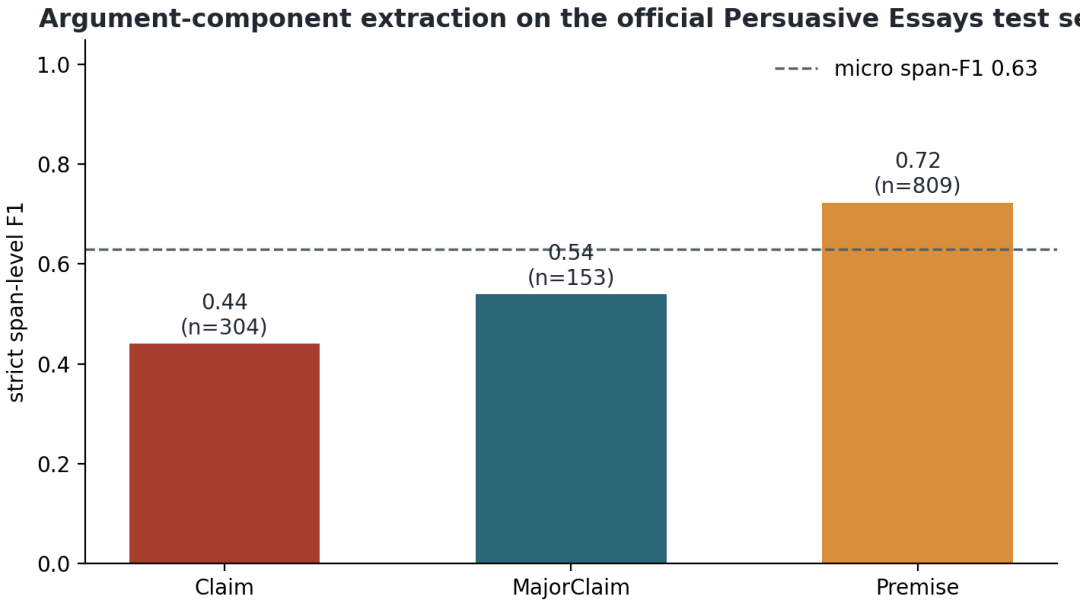


Figure 7.2: Argument-component extraction on the official Persuasive Essays test set, strict span-level F1 per class. Premises are learned well; claim boundaries are the hard case, matching the corpus literature.

The component's second half, pairwise relation classification, is also built (`src/argument_mining/train_relation_classifier.py`, `outputs/relation_classifier.json`). Following the corpus's own structure, candidate pairs are ordered pairs of gold components within one paragraph. DeBERTa reads the source and target components together and decides supports, attacks, or no direct link, trained with class weights on the official split. The results divide along the data's imbalance (Figure 7.3). Supports-links are learned well, F1 0.75 with recall 0.80, which sits where the corpus literature puts link identification when component boundaries are given. Unlinked pairs score 0.95. Attacks are not learned at all (F1 0.0), and the cause is the label supply: attack relations are 0.7 percent of candidate pairs, about the same starvation that capped the Bloom classifier's smallest classes. The macro-F1 of 0.57 against a 0.30 majority baseline is therefore fairly computed but slightly misleading in both directions, pulled down by a class with almost no training signal and pulled up by an easy one. The supports-F1 is the informative number. For the guide, the practical use is ordering. Knowing which premises support which claim tells the lecturer which evidence to probe first. The question generator itself only needs the claims, so this completes the scope's argument-mining specification without changing the pipeline's behaviour.

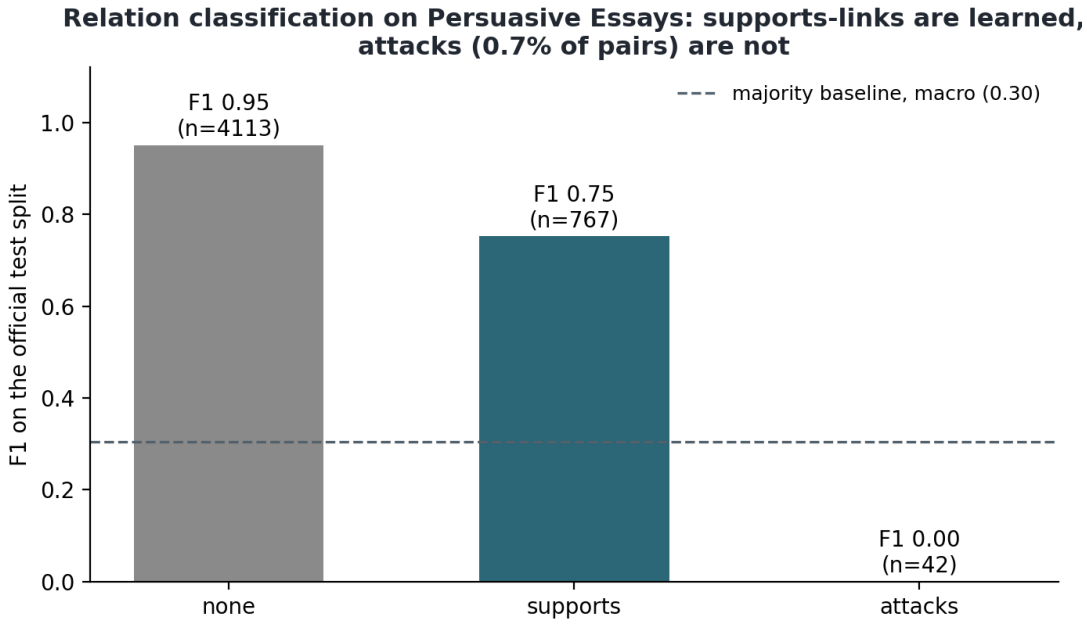


Figure 7.3: Relation classification on the official Persuasive Essays test split, per class. Supports-links reach F1 0.75 with gold components; attack relations, 0.7 percent of pairs, are unlearnable from this corpus, the same label-starvation pattern as the Bloom classifier's smallest classes.

7.7 The assembled guide

With the classifier in place, the output assembler (`src/pipeline/assemble_guide.py`) produces the Verification Interview Guide, the document a lecturer can take into a conversation and the reason the pipeline exists. For a submission it runs the trained detector live and reports the probability with its limits stated in plain language (the in-domain F1, the out-of-domain false-positive risk, and the instruction to treat the score as a reason to talk, never as proof). It lists the SHAP-validated drivers of the decision in lecturer-readable terms. It presents each extracted claim quoted to its exact source sentences. It re-tags every question with the trained Bloom classifier and marks the levels the classifier is weak on as advisory. It closes with a suggested three-level rubric for reading the student's answers. The guide renders to Markdown, Word, and PDF (`outputs/verification_guides/3108a_ai_guide.pdf` is the worked example, generated end to end with live models).

One design note runs through the document. The first page frames the guide as evidence for a conversation, not an accusation, and the fairness results of Chapter 6 made that framing necessary. This was the assembler's first complete version, running the transformer detector and the prompted claims. Section 7.9 returns to it once the trained argument miner and the fine-tuned backend are in place and reports the fully integrated guide.

7.8 Fine-tuning the local backend

Section 7.4 left one question open for my supervisor: whether Backend B, the open-source side of the core comparison, should be the full Llama 3 8B with QLoRA or a smaller model that fits the laptop. The fit probes settled the practical half of it. In 4-bit with a LoRA adapter the 8B loads at 5.3 GB but only trains by spilling past the physical 8 GB into system memory, at 214 seconds per step for a 1024-token sequence. Qwen2.5 3B trains inside VRAM at 2.8 seconds per step with about 3 GB of headroom (`outputs/qlora_fit_probe.json`). On the strength of those probes my supervisor signed off on Qwen2.5 3B as Backend B on 3 July 2026, so the fine-tuning experiment runs on the 3B.

I fine-tuned Qwen2.5-3B-Instruct (Yang et al., 2024) with QLoRA (Dettmers et al., 2023; `src/question_gen/finetune_qg.py`): 4-bit NF4 base, a LoRA adapter (rank 16; Hu et al., 2022) on the attention projections, gradient checkpointing and a paged 8-bit optimiser, batch 1 with gradient accumulation of 8, two epochs over 2,600 passage-to-question pairs from EduQG. The prompt tokens are masked in the loss, so the model learns only to produce the question. Training finished in under an hour on the 4060 at a final loss of 1.42, and the adapter is saved to `models/qg_finetune_qwen3b/`.

The evaluation is built to isolate the fine-tuning effect and nothing else (`src/question_gen/eval_finetune.py`). One fixed set of 18 claims was extracted once from six flagged essays. The base Qwen 3B and the fine-tuned Qwen 3B each wrote verification questions for those same claims, and both sets were scored by the same discrimination simulation used in Chapter

1. The only thing that changes between the two conditions is the question-writing model, so any difference comes from the adapter. The three phases never share the 8 GB. Claims are extracted with the local model through Ollama, which is then unloaded. Each transformer model generates in turn, and scoring loads the embedding model last.

On the raw metric the fine-tune looks like a clear success. The base model's questions score a mean discrimination of 0.033 (95 percent CI 0.015 to 0.051); the fine-tuned model's score 0.154 (95 percent CI 0.085 to 0.227), with a Mann-Whitney p of 0.007 and confidence intervals that do not overlap (Figure 7.3). Taken at face value that is roughly a five-fold gain, and the same 0.15 shows up again at essay scale in Section 8.8.

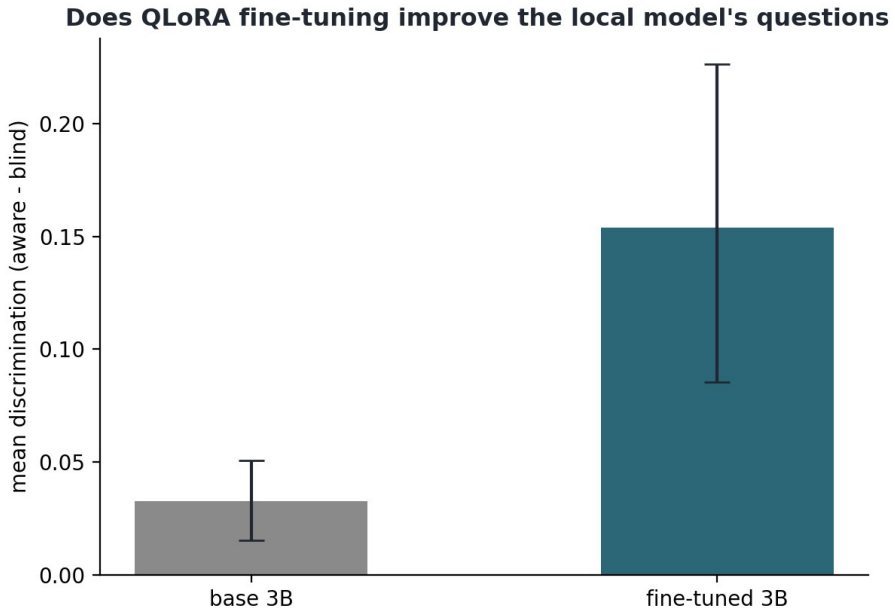


Figure 7.4: The QLoRA fine-tune of Qwen2.5 3B against its own base model on the same 18 claims, mean discrimination with 95 percent bootstrap intervals. The raw score rises from 0.033 to 0.154, but the quality audit in Section 8.9 shows this rise to be an artifact of degenerate output rather than better questions.

It nearly went into the write-up as the headline result of the project. Then I read the questions themselves. The fine-tuned model had overfit the format of its training data. EduQG is largely a multiple-choice corpus, and the adapter learned to emit multiple-choice stems: about 95 percent of its questions are strings like "Which of the following is correct?" or "Which of the following is not a reason why Peugeot is successful?", with no options ever supplied, and a handful are raw JSON fragments leaking from the prompt. These are not verification questions at all. A student could not answer them, and a lecturer could not use them. The full audit across all four models is in Section 8.9, where the base 3B, the Llama 8B and the commercial model each produce zero degenerate questions and the fine-tuned model produces almost only degenerate ones.

The empty stems are also what drove the score up. The literal string "Which of the following is correct?" scores a mean discrimination of 0.44, higher than any real question from any model. A contentless question defeats the simulation. The source-aware and the source-blind answerer both receive a stem with nothing to answer, their responses diverge more or less at random, and the aware-minus-blind gap the metric reports reflects that randomness rather than anything a real student would need the essay for. So the 0.154 measures how empty the fine-tuned model's questions are, not how good they are. The conclusion is the opposite of

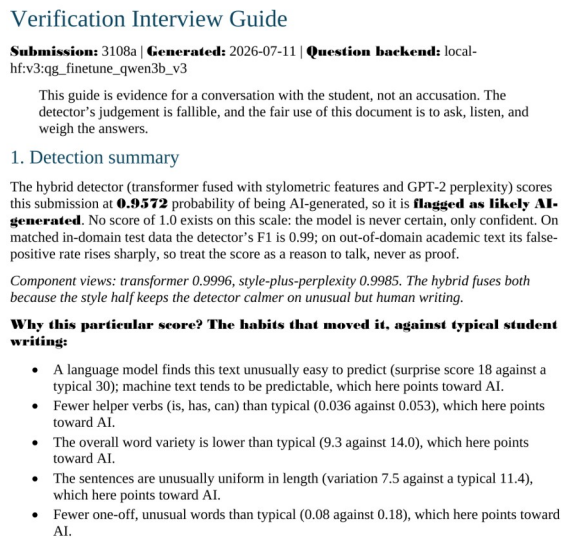
what the raw number suggests. The QLoRA fine-tune on EduQG failed as a question generator for this task.

I keep this in the dissertation instead of deleting it because the failure is informative. It shows that the format of the training data matters more than its size. Two epochs on an overwhelmingly multiple-choice corpus was enough to collapse a capable instruct model into producing multiple-choice stems. More data of the same kind would not help; the fix is the right data, a question-generation set of open-ended questions (or the verification-style prompts themselves), and that is the clear next experiment for Backend B. The failure also exposes a real weakness in the discrimination simulation. The measure assumes well-formed questions and can be gamed by degenerate ones, so it needs a well-formedness guard (a simple filter, or the Bloom classifier, rejecting non-questions) before its scores are trusted. This kind of failure is why the evaluation plan never rested on a single automatic metric, and Section 8.9 turns the episode into that methodological point.

7.9 The integrated guide

Sections 7.5 to 7.8 replaced the first slice's stand-ins one at a time. This section puts the trained pieces together into the final guide, in the form my supervisor approved at our fifth meeting (`src/question_gen/integrated_guide.py`, `src/pipeline/assemble_guide.py`). A submission now flows through five trained models. The prompted extractor supplies each claim in readable phrasing with sentence citations. The trained argument miner of Section 7.6 supplies the provenance in the student's own words: each claim carries the verbatim major-claim, claim and premise spans the miner found in its cited sentences, labelled by role. This is the "spans for provenance, prompt for phrasing" design in one place, and it is stronger than either extractor alone, since the lecturer sees both a sentence they can read and the exact words the model keyed on. The questions are written by the v3 fine-tuned local backend, filtered through the well-formedness gate so nothing degenerate reaches the page (the guide records how many, if any, were dropped), and tagged with the trained Bloom classifier, with the levels the classifier is weak on marked advisory. The detector at the top of the guide is the hybrid of Section 6.7, and it reports its component views. On the worked submission it scores 0.957, noticeably calmer than the transformer's own 0.9996, because the style half pulls the over-confident transformer back. A lecturer facing a possible false positive would want that behaviour.

The document itself is organised for its reader, a lecturer with no time to prepare (Figure 7.5). It opens with the framing the fairness results of Chapter 6 called for, that the guide is there to support a conversation with the student, not to make an accusation. It then gives the detection verdict with its limits in plain language, the SHAP-validated drivers of the decision in lecturer terms, the claims with their two-way provenance and grounded questions, and a three-level rubric for reading the answers. It renders to Markdown, Word and PDF, generated end to end with live models on the 8 GB laptop (`outputs/verification_guides/3108a_ai_guide.pdf` is the worked example). Nothing on the page is a mock-up; every element comes from a trained component of this project. The pipeline as a whole now works, not just the individual parts.



Verification Interview Guide

Submission: 3108a | **Generated:** 2026-07-11 | **Question backend:** local-hf:v3:qg_finetune_qwen3b_v3

This guide is evidence for a conversation with the student, not an accusation. The detector's judgement is fallible, and the fair use of this document is to ask, listen, and weigh the answers.

1. Detection summary

The hybrid detector (transformer fused with stylometric features and GPT-2 perplexity) scores this submission at **0.9572** probability of being AI-generated, so it is **flagged as likely AI-generated**. No score of 1.0 exists on this scale: the model is never certain, only confident. On matched in-domain test data the detector's F1 is 0.99; on out-of-domain academic text its false-positive rate rises sharply, so treat the score as a reason to talk, never as proof.

Component views: transformer 0.9996, style-plus-perplexity 0.9985. The hybrid fuses both because the style half keeps the detector calmer on unusual but human writing.

Why this particular score? The habits that moved it, against typical student writing:

- A language model finds this text unusually easy to predict (surprise score 18 against a typical 30); machine text tends to be predictable, which here points toward AI.
- Fewer helper verbs (is, has, can) than typical (0.036 against 0.053), which here points toward AI.
- The overall word variety is lower than typical (9.3 against 14.0), which here points toward AI.
- The sentences are unusually uniform in length (variation 7.5 against a typical 11.4), which here points toward AI.
- Fewer one-off, unusual words than typical (0.08 against 0.18), which here points toward AI.

Figure 7.5: The first page of the assembled Verification Interview Guide, generated end to end with live models. The detection verdict is the hybrid detector with its component views; the claims that follow carry both readable phrasing and the trained argument miner's verbatim spans; the questions are written by the fine-tuned local backend and tagged by the trained Bloom classifier. The first page presents the guide as support for a conversation with the student rather than an accusation.

Chapter 8: Evaluating the questions (discrimination simulation)

8.1 The idea

A good verification question is one that a student who understands their own work can answer and a student who does not cannot. I evaluate that without humans or an LLM judge, using a discrimination simulation (`src/evaluation/discrimination_sim.py`). For each question, one model answers it WITH the source passage from the essay (the "context-aware" answer) and another answers it with the question ONLY (the "context-blind" answer). I then measure how close each answer is to the source, using embedding similarity (nomic-embed-text; Nussbaum et al., 2024), and take the gap, aware minus blind, as the discrimination score. A high gap means the question genuinely needs the source. A near-zero gap means it can be answered without it. Everything runs on the local model and local embeddings, so the evaluation needs no API key and no judge.

8.2 The first result

I expected my claim-grounded questions to discriminate well. They did not. On the flagged essay, the claim-grounded questions scored a mean discrimination of about 0.05 (95% confidence interval [0.01, 0.10]), while a set of generic essay questions ("what is your main argument", "what evidence most supports your conclusion", "explain your reasoning in your own words") scored about 0.31 ([0.27, 0.36]). The intervals do not overlap, so the difference is real, and it is the opposite of what I assumed (Figure 8.1).

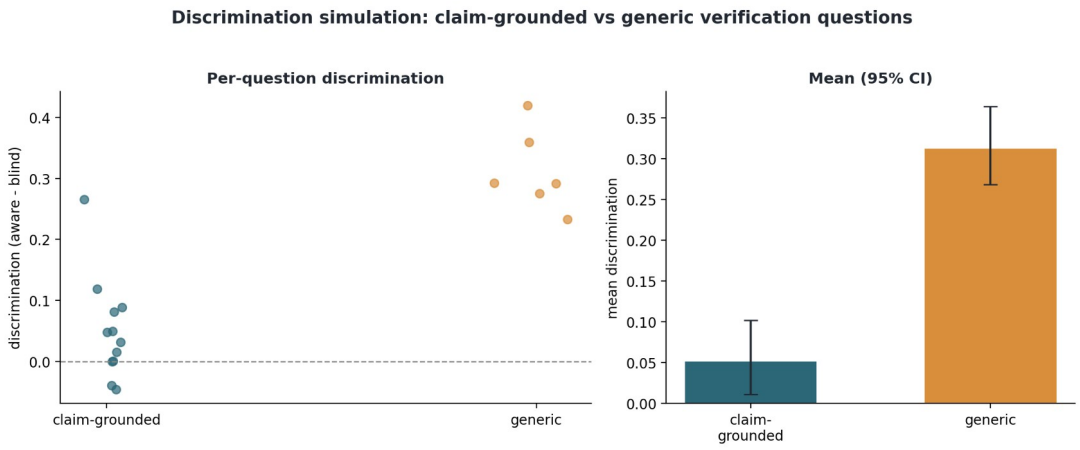


Figure 8.1: Discrimination simulation. Claim-grounded questions (left) cluster low; generic essay questions (right) discriminate much more. The gap is the source-aware minus source-blind answer similarity, with 95% confidence intervals.

8.3 Why the generic questions score higher

The reason is the context-blind model. It is a large language model with broad world knowledge, so a question that names specific content, "what features make Shakespeare's dialogue intense", is answerable from general knowledge without ever reading this student's essay. The source barely changes the answer, so the discrimination is low. A generic question like "what is the main argument of your essay" is different. The blind model has no idea what this particular student argued, so its answer is vague, while the source-aware model answers about the actual essay. The source matters a lot, so the discrimination is high.

So the simulation really measures essay-specificity, whether answering the question requires this student's own essay or only knowledge of the subject. That reframes what a good verification question is. It should force the student to reproduce their own argument, evidence, and choices, rather than recite facts about the topic that anyone knowledgeable could supply. When I rewrote the question-generation prompt to ask for the student's own reasoning and chosen evidence rather than general facts, the grounded score did improve, from about 0.03 to about 0.05, but it stayed well below the generic questions, because a specific question still tends to name the content and hand a knowledgeable answerer most of what it needs.

8.4 A caveat about the stand-in

The context-blind model is a strong stand-in, with far more world knowledge than a student who used AI without understanding the work. A low discrimination score therefore does not mean a question is useless in practice. It means a knowledgeable person who never read the essay can answer it. The simulation is a conservative, hard test of essay-specificity. A real student who did not understand their submission would do worse than the blind model on the grounded questions, so the true picture sits somewhere between the two lines in Figure 8.1.

8.5 What this changes

This is the first measured result for the question generation, and it is more useful as a finding than a score. The strongest verification questions make the student retrieve and justify their own essay: the argument, the specific evidence they chose, and how their points connect. Questions that restate subject facts do not do this. The next iteration of the generator will

blend the claim grounding (so each question is tied to a real passage) with this retrieval-forcing style (so the answer cannot come from general knowledge). The remaining evaluation step is the supplementary LLM-as-judge rubric with cross-model agreement as a second view. The two steps promised here earlier, scaling the simulation across many essays and bringing in the commercial backend, are reported next.

8.6 Commercial versus local: the first scaled comparison

With the simulation in place, I ran the comparison at the centre of the research question. ATU could not provide API access or credits, so the commercial backend uses a free-tier Google Gemini API, plugged in behind the same interface as the local model. The local backend is the same Llama 3.1 8B used throughout. One disclosure on the commercial arm: free-tier quota limits forced a switch of Gemini endpoint partway through (four essays ran on `gemini-2.5-flash`, ten on `gemini-flash-latest`), so the commercial arm is one provider but not one fixed model snapshot. For each essay in a seeded random sample, each backend generated a Verification Interview Guide (three claims, grounded questions with sentence-level provenance), and every question from both backends was scored by the same local discrimination simulation, so what varies is which model wrote the questions.

A small pilot on the single flagged essay from Section 8.2 came first. There, both backends' grounded questions had discrimination intervals that included zero (local 0.032 with [-0.005, 0.083]; commercial 0.022 with [-0.002, 0.048]), too little signal to separate anything. The scaled run below is the real comparison.

Fifteen essays were sampled; fourteen produced a scored guide from both backends (one essay was excluded because claim extraction returned an empty guide; one commercial run was rate-limited for a day and completed on retry), giving each backend 126 questions. The pooled means are local 0.042 with a 95% interval of [0.030, 0.055] against commercial 0.078 with [0.058, 0.098]. Because the same essays sit under both backends, the fair test is paired. The commercial backend scores higher on 10 of the 14 essays. The mean paired difference is 0.036, with a bootstrap interval of [0.008, 0.067]; the paired t-test gives $p = 0.040$ and the Wilcoxon signed-rank $p = 0.030$. Unlike in the pilot, both pooled means sit clearly above zero, so at this scale the grounded questions do measurably need the source (Figure 8.2).

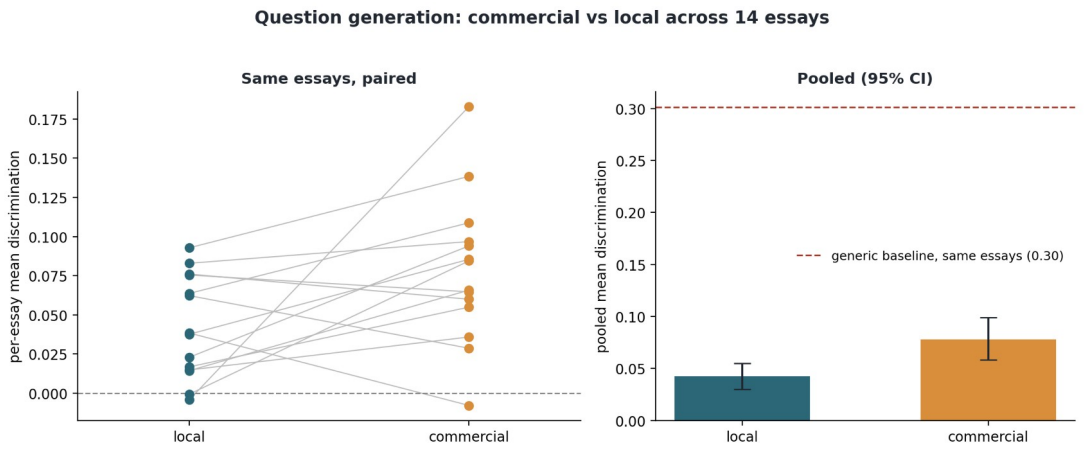


Figure 8.2: Commercial versus local question generation, balanced across the same 14 essays. Left: per-essay mean discrimination, paired. Right: pooled means with bootstrap 95% confidence intervals; the dashed line is the generic-question baseline.

On this measure the commercial backend holds a small but statistically significant advantage, and the local model remains competitive in absolute terms. The gap is about 0.04 against a generic-question baseline of 0.30 with a 95% interval of [0.28, 0.32], now re-measured on the same fourteen essays rather than taken from the pilot (84 generic questions; the pilot's single-essay value of 0.31 barely moved). And the local number comes from a base model that has not yet been fine-tuned. The fine-tuning experiment will test whether the planned QLoRA step closes the gap. The sample-size trajectory is worth recording. Nine essays showed no detectable difference, thirteen sat on the boundary (t p = 0.053), and the full fourteen crossed it. A small-n snapshot could have supported whichever story one preferred, so the dissertation reports the complete set with paired tests. The caveats from earlier still apply: the discrimination measure is conservative, both backends sit well below the generic-question baseline, and the commercial arm is one provider's free tier rather than a frontier model. Extending the comparison to more essays, more providers, and the fine-tuned local backend is mechanical from here, since the framework saves per essay and resumes.

8.7 The LLM-as-judge study

The supplementary evaluation puts LLM judges over the questions with the four-dimension rubric (relevance, specificity, discrimination potential, cognitive appropriateness, each 1 to 5), and the plan has always been to validate judges rather than trust them. All three judges the scope calls for have now rated the twelve grounded questions from the pilot guide: Gemini (2.5 Flash, free tier), Claude (Opus 4.8) and GPT (4o-mini), the last two on a small capped spend approved by my supervisor (`src/evaluation/llm_judge.py`, resumable

across quota interruptions; results and the agreement statistics in `outputs/llm_judge.json`).

The three judges do not tell one story. Gemini and GPT both sit at the ceiling: Gemini rates everything 4.5 to 5.0 (mean 4.81) and GPT even tighter, 4.75 to 5.0 (mean 4.94), so their scores mostly say the questions look well-formed. Claude behaves differently, using the scale from 2.5 to 4.5 (mean 3.67) and marking down the content-naming questions such as "how does the metaphor evoke longing", which general knowledge can answer. Cross-model agreement, the scope's own validation criterion, is poor as a result. Krippendorff's alpha across the three judges is -0.25, and no pairwise rank correlation reaches significance (Spearman 0.20 to 0.45, all $p > 0.13$). With two judges at a ceiling, there is no way for them to agree with a third that discriminates.

Anchoring to the objective measure settles what the disagreement leaves open. None of the three judges correlates positively with the discrimination simulation on the same questions. Gemini's mean rating sits at rho -0.14 ($p = 0.67$), Claude's at -0.30 ($p = 0.34$), and GPT's near-constant ratings manage a significantly negative rho of -0.75 ($p = 0.005$). The few questions GPT rated below the ceiling were among the ones that objectively discriminate best. With $n = 12$ the individual correlations deserve caution (an earlier partial run of the first judge looked strongly negative at seven questions and attenuated to nothing by twelve), but the pattern across three independent providers is consistent. Rubric ratings track how good a question looks. They do not track how well it separates someone who read the essay from someone who did not.

Twelve questions is a thin base for correlations, and Chapter 9 said so, so the study was rerun at five times the size once the scaled comparison of Section 8.12 provided a large pool of scored, well-formed questions to draw from. Sixty questions were sampled evenly from its three fully well-formed arms (twenty each from the local 8B, the base 3B and the fine-tuned v3, one seeded sample that every judge rates identically), and anchored to the same run's per-question simulation scores. The two judges on funded APIs completed the set; the free-tier Gemini judge shares its quota with the comparison itself and is rerun as quota allows. The larger sample settles both open points. First, the anti-correlation is not a small-sample fluke. At sixty questions Claude's mean rating sits at rho -0.34 against the simulation ($p = 0.010$) and GPT's at -0.37 ($p = 0.004$), with GPT's own "discrimination potential" dimension the worst of

all at -0.51 ($p = 0.0001$). Second, the disagreement between judges turns out to be calibration, not noise. The two correlate well in rank (Spearman 0.58, $p < 0.001$), while interval alpha stays negative (-0.54) because GPT's ratings sit at the ceiling (mean 4.87), a full point above Claude (3.56). The two judges share a stable notion of what a good question looks like, and that notion points away from what discriminates. At arm level the cost is concrete: the simulation ranks the sampled arms v3 first (0.071 against 0.039 and 0.017), Claude ranks v3 second, and GPT ranks it last. A backend selection delegated to this panel would have rejected the backend the pipeline ships.

For the evaluation design this is the conclusion the plan anticipated, now with evidence behind it instead of caution. Judge scores in this setup cannot certify question quality on their own. The judges either sit at a ceiling or, when they do discriminate, rank against the objective measure. The judge-free discrimination simulation carries the empirical weight, and the panel's value is this negative result. It is also a second, independent instance of the lesson from Section 8.9: a plausible-looking automatic score, whether a judge rating or a discrimination number, cannot be trusted until it is checked against something that cannot be gamed.

8.8 Like-for-like: four question writers on one fixed claim set

Section 8.6 compared the backends but let each one run its whole pipeline, so the model that chose the claims differed between arms as well as the model that wrote the questions. That leaves an ambiguity. The commercial edge there could come from writing better questions, or simply from Gemini selecting claims that happen to discriminate more. This section removes the ambiguity. One claim set is fixed per essay, extracted once with the neutral local extractor (Llama 3.1 8B), and four question writers answer the same claims: Llama 3.1 8B, free-tier Gemini, the base Qwen2.5 3B, and the QLoRA-fine-tuned Qwen2.5 3B. Every question is scored by the same discrimination simulation, so the only variable across the four arms is the model that writes the question (`src/evaluation/likeforlike_4way.py`, `outputs/likeforlike_4way.json`). The 14 essays are the balanced set from Section 8.6.

On the discrimination score alone the ordering is dramatic (Figure 8.3). The fine-tuned 3B reaches a pooled mean of 0.153 (95% CI [0.105, 0.202]), far above the base 3B at 0.041 [0.028, 0.055], Llama 8B at 0.031 [0.016, 0.045], and commercial Gemini at 0.024 [0.008, 0.041]. Its

paired lead is 0.138 over commercial ($p < 0.001$) and 0.123 over its own base ($p < 0.001$), and it reproduces the 0.154 from Section 7.8. Taken at face value, the locally fine-tuned open model beats the commercial one by a wide margin.

The picture changes once the questions themselves are read, and Section 8.9 covers why. About 95 percent of the fine-tuned model's questions are degenerate multiple-choice stems that game the simulation, so the 0.153 measures how empty they are rather than how good. I set the fine-tuned arm aside for the interpretation here and compare the three models whose questions are well-formed.

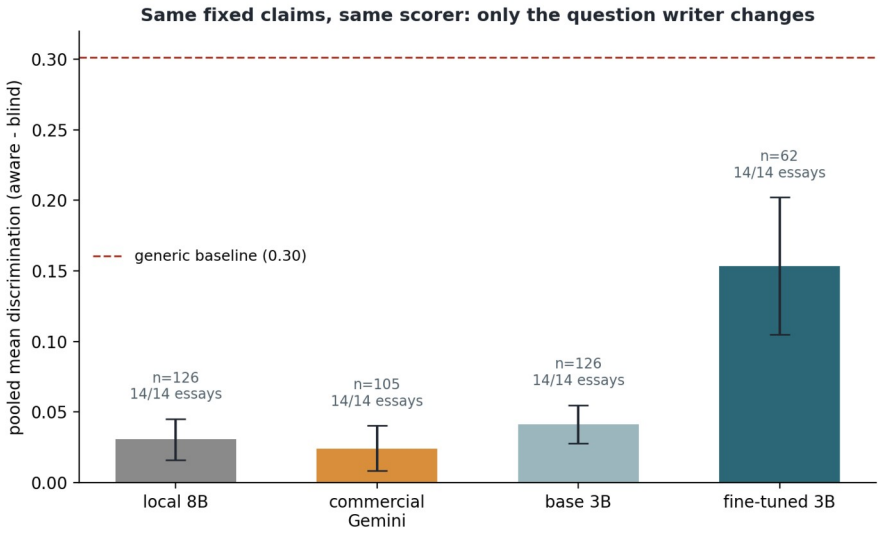


Figure 8.3: The four question writers on one fixed claim set per essay, same scorer, pooled mean discrimination with 95% bootstrap intervals, each bar labelled with its question count and essay coverage. The fine-tuned 3B posts the highest raw score, but Section 8.9 shows that score to be an artifact of degenerate questions; the meaningful comparison is among the other three arms.

Among those three the picture is modest and close. On the fixed claims the three models are statistically indistinguishable: base Qwen 3B at 0.041, Llama 8B at 0.031, and Gemini at 0.024, all low and with overlapping intervals. The run initially covered only 9 of 14 essays on the commercial arm because free-tier quota ran out mid-run; the framework resumed when quota returned, and the numbers here are the complete set (105 commercial questions across all 14 essays, a few claims lost to rate-limiting inside otherwise-covered essays). The complete set settles what the partial run could only suggest. In Section 8.6, with each backend choosing its own claims, Gemini held a significant edge over Llama 8B (0.078 to 0.042, paired $p = 0.040$). Here, on identical claims, that edge is gone (paired difference -0.005, $p = 0.62$, Gemini higher on seven of fourteen essays, a coin flip). The Section 8.6 advantage does not survive fixing the claims. I take this to mean the advantage came substantially from claim

selection, with Gemini choosing easier-to-discriminate claims when allowed to, and not from writing stronger questions on the same ones. The design here cannot rule out every alternative, but the disappearance is now measured on the full sample rather than hypothesised. The firm conclusion of this section is narrower than the one I first drafted, and arguably more useful. When the task is fixed, small open models running on a laptop write questions as discriminative as a commercial model's. The second half of the research question, whether a fine-tuned local model can beat a commercial one, stays open at this point, because the fine-tune that was supposed to answer it produced unusable questions.

8.9 A quality audit of the generated questions

Section 8.8 leaned on a number without reading what it scored. This section reads the questions, and the gap between the two is the biggest methodological lesson in the project (`src/evaluation/qg_quality_audit.py`, `outputs/qg_quality_audit.json`). Every question from all four arms was checked with a transparent rule for degeneracy: multiple-choice stems (which never carry options here), raw JSON fragments leaking from the prompt, and contentless "which is correct" forms.

Figure 8.4 shows the result. The Llama 8B, base 3B and commercial arms produce no degenerate questions at all. The fine-tuned 3B produces almost nothing else: 59 of its 62 questions, just over 95 percent, are stems like "Which of the following is correct?" or "Which of the following is not a reason why Peugeot is successful?", with no options supplied. The fine-tune, two epochs on the largely multiple-choice EduQG corpus, had overfit the format of the training data and forgotten how to write an open question. None of these are usable in a verification interview.

The second panel matters for the evaluation method. The empty stems do not just pass the discrimination simulation, they outscore the real questions. The single literal string "Which of the following is correct?", scored eleven times across the run, averages a discrimination of 0.44, higher than the mean of every model's real questions. The reason is structural. A contentless question gives the source-aware answerer and the source-blind answerer nothing to latch onto, so their answers wander apart for reasons unrelated to the source, and the aware-minus-blind gap the metric rewards is large. The simulation assumes the questions it

scores are well-formed. A degenerate generator can exploit that assumption and post an arbitrarily high score.

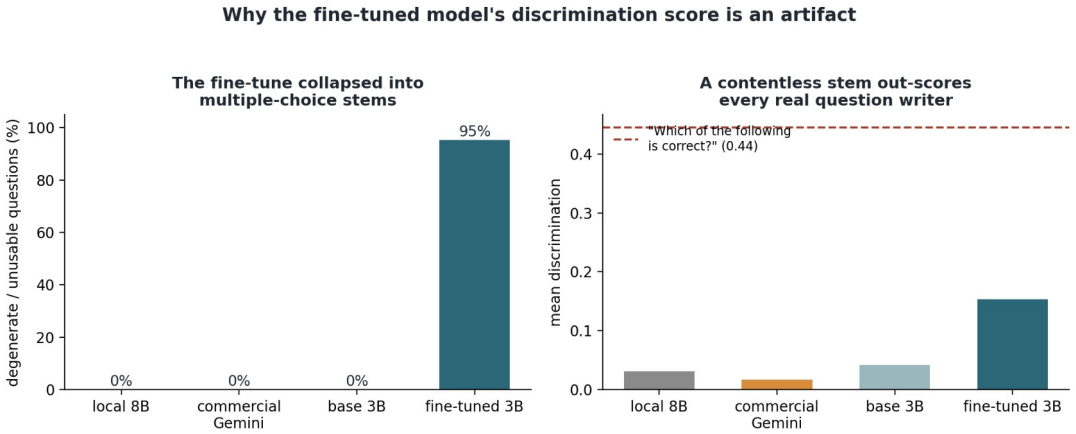


Figure 8.4: Auditing question quality behind the discrimination score. Left: the share of degenerate, unusable questions per model. Right: the mean discrimination of each model's questions against the score of the contentless stem "Which of the following is correct?" (dashed line), which out-scores every real question writer.

Two conclusions follow. For Backend B, the format of the training data dominates, so fixing the fine-tune means the right data rather than more data: an open-ended question-generation set (SQuAD-style questions, or the pipeline's own verification prompts distilled into training pairs) instead of a multiple-choice corpus. Section 8.10 runs that experiment. For the evaluation, the judge-free discrimination simulation, valuable as it is for well-formed questions, needs a well-formedness gate in front of it, and no automatic score should be read without inspecting the text behind it. That gate is now part of the pipeline (`src/question_gen/wellformed.py`). The same transparent rule the audit uses is applied wherever questions leave the system, the guide builder drops malformed questions before a lecturer can see them and records how many it dropped, and every fine-tune evaluation reports its degeneracy rate next to its score. The evaluation plan required reading outputs before trusting scores, and that requirement is why this failure was caught here rather than in a viva.

8.10 Fixing the fine-tune: v2 on open-ended data

Section 8.9 made a testable prediction: if the degeneracy was caused by EduQG's multiple-choice format, then changing only the training data to open-ended questions should fix it. So I re-ran the fine-tune with everything held identical except the data, swapping EduQG for SQuAD (Rajpurkar et al., 2016), whose questions are open-ended and passage-grounded (`src/question_gen/finetune_qg_v2.py`, adapter in

`models/qg_finetune_qwen3b_v2/`). The evaluation repeats the Section 7.8 design on the same 18 claims, base against the new adapter, and it audits degeneracy before reading any score (`src/question_gen/eval_qg_v2.py`, `outputs/qg_v2_eval.json`).

The prediction held. The v2 model's questions are 0 percent degenerate against v1's 95, so the cause was the format of the training data and not fine-tuning as such (Figure 8.5). This time the discrimination gain is real rather than an artifact, because the questions are well-formed. On the same claims, v2 scores a mean discrimination of 0.102 (95% CI [0.060, 0.144]) against the base model's 0.027 ([0.015, 0.040]), a Mann-Whitney p of 0.0003. The contrast with v1 matters here. v1's higher 0.154 came from empty multiple-choice stems gaming the metric. v2's lower 0.102 comes from genuine questions that need the source more than the base model's do.

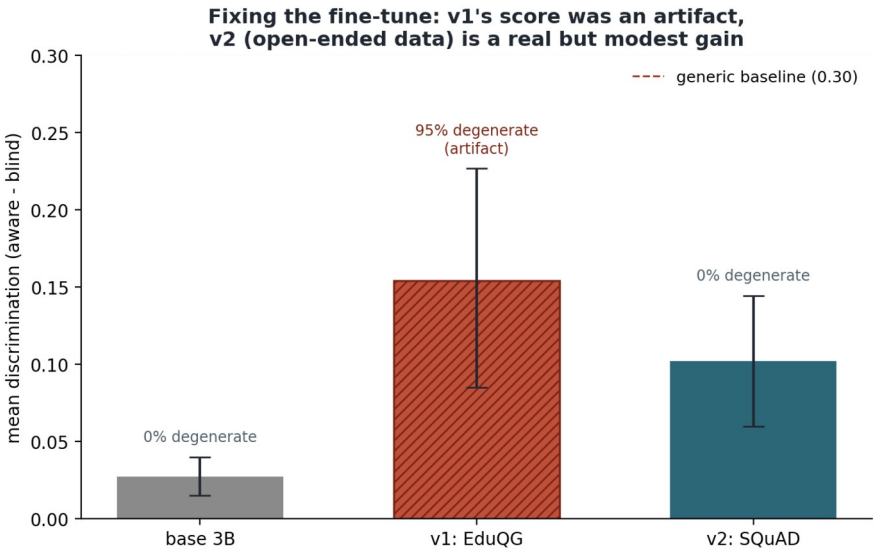


Figure 8.5: The fine-tune, corrected. Base 3B, the v1 EduQG adapter (hatched: its score is an artifact of 95 percent degenerate questions), and the v2 SQuAD adapter, mean discrimination on the same 18 claims with 95 percent intervals and each bar's degeneracy rate. v2 is a real gain over base (p = 0.0003) on well-formed questions, but modest and still below the generic-question baseline.

v2 is a partial success, not a triumph, and I hold the interpretation to what was measured. Its 0.102 is well under the generic-question baseline of about 0.30, and reading its questions shows why. SQuAD trains a factual style ("What is the purpose of judicial review?"), and factual questions are partly answerable from general knowledge, the property Section 8.3 showed the simulation penalises. Two smaller points. v2 needed a cleaned output parser, because the adapter sometimes appended JSON fragments after the question that an earlier extractor would have kept; the parser now takes the first well-formed question and drops the

rest. And the degeneracy audit ran before the score, as the standing rule now requires. Fine-tuning a small local model does help once the training data has the right form. But SQuAD's factual questions are not the ideal signal for verification, so the next step is a training set of reasoning-demanding questions, distilled from the pipeline's own verification prompts. That is the v3 experiment of the next section. At this stage Backend B was a diagnosed and partly fixed component, and it is reported that way rather than as the false win of Section 7.8's first draft.

8.11 Completing the data-format experiment: v3 on the pipeline's own style

The third leg trains on data in the format the task itself uses. A training set of 2,604 verification questions was distilled from the pipeline itself (`src/question_gen/build_v3_dataset.py`): the local Llama 3.1 8B ran the production claim-extraction and question-writing prompts over 307 training essays, and each well-formed question became one passage-to-question pair, with the passage being the claim plus its source sentences, which is what the model sees at inference. The training set needed some care. All fifteen evaluation essays were excluded from the training pool, so nothing the fine-tunes are tested on was ever trained on. Only questions passing the well-formedness gate became training targets, since a noisy teacher would poison the experiment. And the teacher is local rather than a commercial API because Llama's licence permits training other models on its output. Training kept every setting identical to v1 and v2 once again (`src/question_gen/finetune_qg_v3.py`); the final loss of 0.72, against 1.42 for v1 and 1.27 for v2, already suggests that on-format data is easier to learn.

The evaluation follows the standing protocol: degeneracy audited before any score, same 18 fixed claims (`src/question_gen/eval_qg_v3.py`, `outputs/qg_v3_eval.json`). v3 is fully well-formed, zero degenerate questions out of 42, and it answers the request for three questions per claim far better than v2 did (2.3 per claim against 1.3). Its output reads like the pipeline's intended style: "How did you decide to use the example of a decision made by a public authority as evidence", "what role does the concept of authority play in your argument, and how did you connect it", "in what ways does your discussion relate to the broader themes of your essay". On the metric it scores a mean discrimination of 0.064 (95% CI [0.033, 0.096]), roughly two and a half times the base model's 0.027 ([0.015, 0.040]), with the intervals barely

brushing. That also doubles the 8B teacher's own score in the fixed-claim comparison (0.031, Section 8.8), so the 3B student overtook the model that taught it (Figure 8.6).

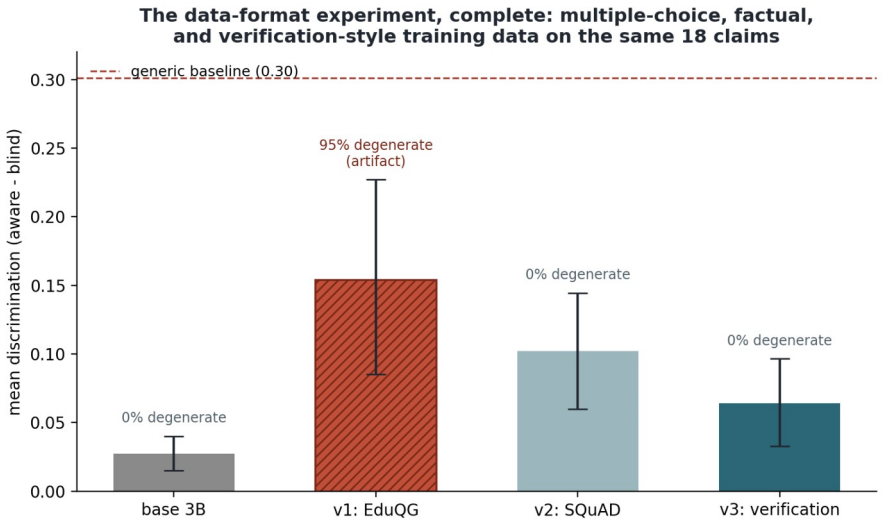


Figure 8.6: The data-format experiment complete on the same 18 claims: base model, v1 (multiple-choice EduQG, hatched because its score is a degeneracy artifact), v2 (factual SQuAD) and v3 (verification-style self-distillation), with each bar's degeneracy rate. Both open-ended formats produce fully well-formed output and real gains; only the multiple-choice format collapses.

One result does not fit a tidy ranking. v2 still posts the higher number (0.102 against 0.064), and the reason matters more than the numbers themselves. The two adapters learned different trades. v2 writes one terse factual question per claim; that style happens to suit the simulation, but "What is the purpose of judicial review?" is not a verification question, since it never asks the student about their own essay. v3 writes the questions the product actually needs, demanding the student's reasoning, choices and connections, but in doing so it quotes the claim's content back, and Section 8.3 established that handing content to the source-blind answerer shrinks the measured gap. The metric, taken alone, would therefore pick the adapter whose output is unfit for purpose. This is the third time in this chapter that a single automatic number would have led to the wrong choice, after the v1 artifact and the judge panel, and it is why the decision about Backend B is not delegated to the metric. v3 becomes the working local backend because its output is well-formed, on-style and a real improvement over base. v2's higher raw score is documented and explained rather than chased.

The completed experiment is the dissertation's answer on fine-tuning. The format of the training data dominates: a multiple-choice corpus collapses the model, and either open-ended format fixes it. Fine-tuning a 3B model on an 8 GB laptop reliably lifts it well clear of its base, whichever open format is used, and self-distillation can lift a small student past its own

larger teacher on the target measure. No single automatic score, this one or a judge's, is allowed to settle a design decision by itself in this project.

8.12 The comparison at scale: thirty essays

Chapter 9 names the comparison's sample size as a limitation, so the final experiment scales the fixed-claim design from fourteen essays to thirty. The arms are v3 (the backend the pipeline actually ships), the local 8B, the base 3B, and commercial Gemini (`src/evaluation/likeforlike_scaled.py`, `outputs/likeforlike_scaled.json`). The protocol does not change: one fixed claim set per essay from the neutral extractor, every writer answering the same claims, one scorer, and the degeneracy rate reported next to every score. Contamination needed guarding against at this size. The sixteen new essays come only from the pool that is neither an evaluation essay nor one of the 307 essays that supplied v3's training questions. And the original fourteen essays keep their claims and questions as first generated; nothing is regenerated. The run scored 901 questions across the four arms. None is degenerate.

The headline result is the strongest in this dissertation (Figure 8.7). The fine-tuned v3 reaches a mean discrimination of 0.085 (95% CI [0.071, 0.100]), more than double every other arm. Against its own base model the paired difference is +0.046 ($p < 0.0001$, higher on 25 of 30 essays). Against the commercial model, on the 29 essays both arms cover, it is +0.040 ($p = 0.0003$, higher on 24 of 29). On this task, with the claims held fixed, a model fine-tuned on one consumer laptop writes measurably better verification questions than the commercial free tier. The comparison also replicates the earlier null. Commercial Gemini has no edge over the plain local 8B (paired difference +0.003, $p = 0.74$, $n = 29$), which repeats the Section 8.8 finding that the commercial advantage in Section 8.6 came from claim selection, not question writing.

I trust this result for a specific reason, having retracted a version of it once before. The v1 "win" in Section 8.9 came from degenerate output that gamed the metric. Here every one of v3's 184 questions passes the well-formedness gate, and reading them confirms they are ordinary verification questions in the intended style. The score and the output quality agree this time. Section 8.9 established that agreement as the condition for believing any number this pipeline produces.

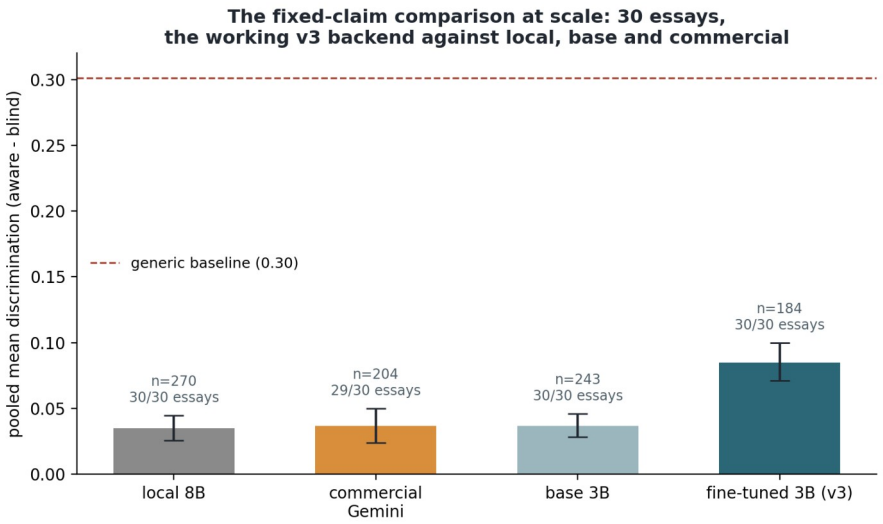


Figure 8.7: The fixed-claim comparison at thirty essays with the working v3 backend. All 901 questions are well-formed. v3 clears the base 3B ($p < 0.0001$) and, on the 29 shared essays, the commercial model ($p = 0.0003$); the commercial-versus-8B null replicates. Coverage is printed on each bar: the free-tier commercial arm covers 29 of 30 essays.

Two disclosures apply. The commercial arm is a free-tier flash model, not a frontier one, so the claim is "a QLoRA fine-tune on an 8 GB laptop beats the free commercial tier at writing verification questions", not "local models beat commercial models". And quota interruptions shaped the commercial arm's coverage: it spans 29 of the 30 essays and mixes two versions of the same model, fourteen essays on gemini-2.5-flash (105 questions) and fifteen on gemini-flash-latest (99 questions) across two refills. The refilled questions are shorter (median 30 words against 43) and score twice as high (0.050 against 0.024), so the mixture strengthens the commercial arm rather than the local side, and v3 clears it anyway. With both stated, the second half of the research question has its answer. A locally fine-tuned open model can match, and in this comparison beat, the commercial backend it was allowed to compare against.

Chapter 9: Discussion

9.1 Answering the research question

The research question had two halves. The first asked whether an explainable pipeline can support lecturers in academic integrity verification by combining transparent detection with argument-aware question generation. The second asked how far locally fine-tuned open-source models can match commercial LLMs at the task. Both halves now have concrete answers, though neither is the simple answer I expected at the start.

On the first half, the pipeline exists and works end to end. A submission goes in and a detector trained on a matched, audited corpus scores it. The score arrives with a faithfulness-tested explanation written in lecturer language instead of a bare percentage. The submission's own claims are extracted with sentence-level provenance. Verification questions grounded in those claims are generated, gated for well-formedness, tagged with a cognitive level, and assembled into an interview guide. The guide's first page frames what follows as evidence for a conversation with the student, not an accusation, and the rest of the design takes the same position.

Every stage was evaluated, and the evaluations are what make this answer credible. In domain the detection is excellent, an F1 of 0.990 with a bootstrap interval of roughly [0.97, 1.00]. The detector also transfers across unseen generators at 0.97. Across text domains it degrades to 0.79, and it degrades in the direction that matters most, by falsely flagging human academic writing. The explanation layer went through the same scrutiny. All three token-level methods, attention included, fail a faithfulness test that the feature-level SHAP account passes, so the lecturer sees the feature account.

On the second half, the strongest result in the dissertation is where the comparison ended up. At thirty essays the fine-tuned 3B holds its gain over its own base (+0.046, $p < 0.0001$, higher on 25 of 30 essays) and beats the free-tier commercial model on the 29 essays both cover (+0.040, $p = 0.0003$, higher on 24 of 29), with every question well-formed. I make that claim after retracting two weaker forms of it. On this evidence the answer to "can locally fine-tuned open models match commercial LLMs at this task" is yes, and on the fixed task the fine-tuned local model does better than the free commercial tier.

Getting there depended on holding the task fixed, and that dependence is a finding in itself. When each backend chose its own claims, the commercial model held a small significant edge (paired difference 0.036, p = 0.040). When every model wrote questions for identical claims, the edge vanished (difference +0.003, p = 0.74). Most of the apparent commercial advantage was claim selection, not question writing. Fine-tuning then moved the local backend well clear of its own base once the training data had the right format.

There are caveats. The commercial arm is a flash model, not a frontier one. All well-formed writers still sit below the generic-question baseline. And because the comparison's outcome moved with the experimental design, the dissertation reports every design it ran, not only the friendliest one.

9.2 What the internal checks caught

The project's thesis is that unexplained scores cannot be trusted. Three of my own scores demonstrated the point, and the checks that caught them are among the most useful results in the dissertation.

The first case was the fine-tune artifact. The v1 adapter posted a discrimination score four times its base model, with non-overlapping confidence intervals, and for about an hour it looked like the headline result of the project. Reading the actual output showed 95 percent degenerate multiple-choice stems. The audit then showed that those stems win the metric, because a contentless question makes the source-aware and source-blind answers diverge at random.

The second case was the judge panel. Three commercial judges, the validation the scope called for, produced ratings that neither agree with each other (Krippendorff's alpha of -0.25) nor track the objective measure. Rerunning the panel at sixty questions made the verdict firmer. At sixty the judges agree with each other in rank, and both funded judges anti-correlate significantly with the simulation. Rubric ratings measure how good a question looks. The panel study stands as a replicated negative result.

The third case was quieter but had the same shape. Ranking the fine-tunes by raw discrimination would have picked v2, whose terse factual one-liners are not verification questions at all, over v3, whose output is what the product needs.

One methodological posture covers all three cases, and it is built into the pipeline now rather than left as advice. Every automatic score is anchored to something it cannot game. The judges are anchored to the simulation. The simulation is gated for well-formedness, and the degeneracy rate is reported next to every score. No design decision was delegated to a single number. That posture was demonstrated three times on the project's own results, and I would argue it is as much a contribution as any component, since the field's detector vendors do not practise it.

9.3 Fairness and verification

The fairness evidence influenced the design more than any other single result. The literature reports that detectors flag non-native English writers at wildly disproportionate rates, and my own robustness tests reproduced the mechanism behind that pattern at domain level. Human arXiv abstracts were falsely flagged 79 percent of the time, because dense academic prose looks, to these features, like machine text. A detector with that failure mode cannot be the end of any fair process. The pipeline therefore treats detection as the opening of a conversation and moves the evidential weight to verification. The questions are ones a student who wrote and understood the work can answer and an impostor cannot. Each question is traceable to exact sentences of the submission, so nothing is invented.

The discrimination simulation made clear what such questions have to look like. Its central finding is that content-naming questions are answerable from general knowledge. Good verification questions instead force the student to reconstruct their own reasoning, name their own evidence, and show how their own argument fits together. That finding held in every later experiment. I rewrote the generation prompt around it and built the v3 training data on it, and it explains why v2 and v3 rank the way they do on the raw metric. It also sets a limit on what the simulation can measure. The context-blind model is a much stronger stand-in than a student who submitted work they do not understand, so measured discrimination is a conservative floor. It is not an estimate of classroom performance.

9.4 Limitations

The limitations are stated where they arise in the chapters. This section gathers the important ones in one place.

The evaluation of question quality rests on a simulation, not on students. That was a deliberate scope decision (it keeps the project clear of human-participants approval), but it means the discrimination scores are proxies with a conservative bias. Classroom validation is the single most important piece of future work.

Sample sizes grew but stay modest. The final comparison runs thirty essays with 901 questions, and the fine-tune tests rest on 27-essay paired designs. The judge study grew from twelve questions to sixty. At sixty, its anti-correlation with the simulation could no longer be attributed to sample size. Bootstrap intervals are used throughout, and the sample-size trajectory of the first comparison is reported in full because small snapshots would have supported whichever story I preferred.

The detection corpus pairs one human corpus with one generator. The detector transfers to six unseen generators, but the corpus itself cannot show generator diversity. The commercial arm is one provider's free tier rather than a frontier model, and quota limits caused disclosed irregularities. On the home corpus the detection task saturates. Because of that, the training-distribution control could show the balancing is harmless, but not whether it helps on a harder task.

The argument-mining component extracts claims at a working but unspectacular span-F1 of 0.63. The relation classifier learns supports-links well (F1 0.75 with gold components) but cannot learn attack relations from the 0.7 percent of pairs that carry them. That is a label-supply ceiling, and the Bloom classifier hits the same one.

9.5 Implications for practice

For a lecturer, the practical output of this work is a defensible process, not a score. A flag arrives with named, tested reasons ("the sentences are unusually uniform, the vocabulary less varied than a typical student's") and with an interview guide instead of a verdict.

For an institution, two of the results carry direct consequences. Detectors, including good ones, must not be pointed at text from domains they were not trained on, and any deployment needs domain checks, calibrated thresholds and an abstain band, because the failure mode is false accusation and it lands unevenly. The local-versus-commercial result removes a dependency. Models that run on a single consumer laptop matched a commercial API on the fixed task. The pipeline was built entirely under that constraint, and the constraint

turned out to be an advantage, because it means an institution can run verification without sending student work to a third party. Given that submissions are personal data, that is a real consideration.

Chapter 10: Conclusions and future work

10.1 What this dissertation contributes

The project set out to give lecturers a process they can defend in place of a single unexplained score, and to find out whether that process needs a commercial API behind it. The short answer to both questions: the process exists and runs, and it does not need the API. Each contribution below is backed by a result in the preceding chapters.

First, the working system. A Verification Interview Guide generates end to end from a submission, with live models, on an 8 GB laptop. The guide presents its contents as starting material for a conversation with the student, not as a verdict.

Second, a matched detection corpus and an audited detector. The corpus pairs 640 real student essays with 640 AI essays matched on topic and length, so the two classes differ in writing and not in anything a detector could use as a shortcut. When the first detector scored perfectly, an audit traced the score to a corpus artefact, removed it from both classes, and re-established the result on style alone: F1 0.99 in domain, and 100 percent detection of essays from two commercial generators the model never saw in training, with no false accusations of the matching human essays. The audit procedure is reusable beyond this project.

Third, explanations that have been tested rather than assumed. Three attribution methods were measured on one ablation yardstick. The two token-level methods fail it, both with the same diffuse signal, and the feature-level SHAP account passes, so that account is what a lecturer sees. In this dissertation, "this explanation is faithful" is an empirical statement.

Fourth, question generation with provenance built in. Claims cite sentence numbers, the sentence text is looked up from the submission itself, and a generated question can always be traced to exact lines. A well-formedness gate keeps degenerate output away from the lecturer and away from the metrics.

Fifth, the local-versus-commercial answer. The comparison ran under two designs, and the outcome depended on the design, which is itself a finding. With self-chosen claims the commercial backend held a significant edge. With the claims held fixed the edge disappeared, and after fine-tuning, the local 3B wrote better questions than the free commercial tier on 24

of the 29 shared essays ($p = 0.0003$) at thirty-essay scale. A university can run this kind of verification on ordinary hardware, without sending student work to a third party.

Sixth, a controlled data-format experiment. The same model was fine-tuned three times with identical settings, varying only the training corpus. A multiple-choice corpus collapsed the model into unusable output, either open-ended corpus repaired it, and the verification-style data produced the backend the pipeline ships. In the same runs, the 3B student overtook its 8B teacher on the target measure.

Seventh, an evaluation method for verification questions that needs no judge, together with a demonstration of how it can be gamed. The discrimination simulation measures whether a question needs the source. Contentless questions defeat it, and the well-formedness gate closes that hole. The discovery and the gate together are a methods contribution to an evaluation literature that leans heavily on LLM judges.

Eighth, a replicated negative result on those judges. At twelve questions, three commercial judges neither agreed with each other nor tracked the objective measure. Rerun at sixty questions, the two funded judges agree with each other in rank and still point away from the measure, and both rank the best backend near the bottom. That is direct evidence for anchoring judge studies to an objective measure instead of trusting the panel.

10.2 Future work

The next step with the most to offer is a study with real students, run under ethics approval, to test whether the generated questions separate authors from non-authors in practice as the simulation predicts. This project excluded human participants by design, so that ground is untested. The simulation's conservative bias gives the study a fair chance of confirming the prediction, and the guide is the ready-made instrument for it.

Several extensions follow from measured limits. The commercial arm should widen to multiple providers and a frontier model, now that the framework treats any API as a plug-in backend. The detection corpus gained test essays from two more model families during the robustness work; the training side should be widened the same way, so the transfer results can be shown on the corpus itself without leaning on an external benchmark. Cross-domain deployment needs calibrated per-domain thresholds before anyone should trust a flag outside student essays, and the measured abstain band shows what calibration alone will not

fix. The training-distribution control should be re-run on a task hard enough to have resolution. The Bloom classifier is held back by label supply, not architecture, so a better-populated labelling of higher-order questions would lift the guide's quality control. The relation classifier learns supports links well (F1 0.75) but attack relations are too rare in the training corpus to learn at all; a corpus with denser attack annotation would finish that component. And the agreed extension of the classification target, a third partial-AI class, becomes worth piloting once mixed human-and-AI documents can be constructed carefully, since partial assistance is the realistic case in coursework.

10.3 Closing

One rule ran through this work: do not present a number you cannot explain, and do not trust a number you have not tried to break. Applying that rule to my own results removed three headline claims, and each time the finding that replaced the headline was more useful and easier to defend. The version of the local-beats-commercial claim that survives in Chapter 8 is the one that earned its place. Where the pipeline can show its working it can be trusted, and where it cannot, the document says so.

References

Alsop, S. and Nesi, H. (2009). Issues in the Development of the British Academic Written English (BAWE) Corpus. Corpora, 4(1), pp. 71 to 83.

Anderson, L. W. and Krathwohl, D. R. (2001). A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. Longman, New York.

Dettmers, T., Pagnoni, A., Holtzman, A. and Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2305.14314.

Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT 2019), pp. 4171 to 4186.

DeYoung, J., Jain, S., Rajani, N. F., Lehman, E., Xiong, C., Socher, R. and Wallace, B. C. (2020). ERASER: A Benchmark to Evaluate Rationalized NLP Models. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020), pp. 4443 to 4458.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783.

Guo, S., Liao, X., Li, C. and Chua, T.-S. (2024). A Survey on Neural Question Generation: Methods, Applications, and Prospects. In Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024). arXiv:2402.18267.

Hadifar, A., Bitew, S. K., Deleu, J., Develder, C. and Demeester, T. (2022). EduQG: A Multi-format Multiple Choice Dataset for the Educational Domain. IEEE Access. arXiv:2210.06104.

Hayes, A. F. and Krippendorff, K. (2007). Answering the Call for a Standard Reliability Measure for Coding Data. Communication Methods and Measures, 1(1), pp. 77 to 89.

He, P., Liu, X., Gao, J. and Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. In Proceedings of the 9th International Conference on Learning Representations (ICLR 2021).

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. In Proceedings of the 10th International Conference on Learning Representations (ICLR 2022). arXiv:2106.09685.

Jain, S. and Wallace, B. C. (2019). Attention is not Explanation. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2019), pp. 3543 to 3556.

Jin, Y., Yan, L., Echeverria, V., Gasevic, D. and Martinez-Maldonado, R. (2024). Generative AI in Higher Education: A Global Perspective of Institutional Adoption Policies and Guidelines. arXiv:2405.11800.

Krishna, K., Song, Y., Karpinska, M., Wieting, J. and Iyyer, M. (2023). Paraphrasing Evades Detectors of AI-Generated Text, but Retrieval is an Effective Defense. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2303.13408.

Kumar, S., Gulwani, R. and Singh, P. (2025). Automated Analysis of Learning Outcomes and Exam Questions Based on Bloom's Taxonomy. arXiv:2511.10903.

Liang, W., Yuksekgonul, M., Mao, Y., Wu, E. and Zou, J. (2023). GPT Detectors Are Biased Against Non-Native English Writers. Patterns, 4(7), 100779. arXiv:2304.02819.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017), pp. 4766 to 4777.

Mindner, L., Schlippe, T. and Schaaff, K. (2023). Classification of Human- and AI-Generated Texts: Investigating Features for ChatGPT. arXiv:2308.05341.

Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D. and Finn, C. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023), PMLR 202, pp. 24950 to 24962.

Mohammadshahi, A., Scialom, T., Yazdani, M., Yanki, P., Fan, A., Henderson, J. and Saeidi, M. (2023). RQUGE: Reference-Free Metric for Evaluating Question Generation by Answering the

Question. In Findings of the Association for Computational Linguistics: ACL 2023, pp. 6845 to 6867. arXiv:2211.01482.

Nussbaum, Z., Morris, J. X., Duderstadt, B. and Mulyar, A. (2024). Nomic Embed: Training a Reproducible Long Context Text Embedder. arXiv:2402.01613.

Oketch, K., Lalor, J. P., Yang, Y. and Abbasi, A. (2025). Bridging the LLM Accessibility Divide? Performance, Fairness, and Cost of Closed versus Open LLMs for Automated Essay Scoring. arXiv:2503.11827.

Pietron, M., Olszowski, R. and Gomulka, J. (2024). Efficient Argument Classification with Compact Language Models and ChatGPT-4 Refinements. arXiv:2403.15473.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI technical report.

Rajpurkar, P., Zhang, J., Lopyrev, K. and Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. In Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP 2016), pp. 2383 to 2392.

Stab, C. and Gurevych, I. (2017). Parsing Argumentation Structures in Persuasive Essays. Computational Linguistics, 43(3), pp. 619 to 659.

Sundararajan, M., Taly, A. and Yan, Q. (2017). Axiomatic Attribution for Deep Networks. In Proceedings of the 34th International Conference on Machine Learning (ICML 2017), PMLR 70, pp. 3319 to 3328.

Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., et al. (2023). M4: Multi-Generator, Multi-Domain, and Multi-Lingual Black-Box Machine-Generated Text Detection. arXiv:2305.14902.

Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., Mohammed Afzal, O., Mahmoud, T., Puccetti, G., Arnold, T., et al. (2024). SemEval-2024 Task 8: Multidomain, Multimodel and Multilingual Machine-Generated Text Detection. In Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024), pp. 2057 to 2079. arXiv:2404.14183.

Wu, J., Yang, S., Zhan, R., Yuan, Y., Chao, L. S. and Wong, D. F. (2025). A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions. Computational Linguistics, 51(1), pp. 275 to 338. arXiv:2310.14724.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., et al. (2024). Qwen2.5 Technical Report. arXiv:2412.15115.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E. and Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track. arXiv:2306.05685.

Appendices

Appendix A: the prompts the pipeline uses

The pipeline's behaviour depends on its prompts, so this appendix reproduces them verbatim. Three of them do the argument-aware work.

Claim extraction (system role): "You analyse a student essay for an academic-integrity verification interview. A claim is a substantive position the student asserts and argues for, not background, definition, or quotation. You cite the sentence numbers each claim is drawn from. Reply with JSON only." The user turn supplies the essay as numbered sentences and asks for the n most important claims, each with a one-line paraphrase and its sentence numbers. The model only returns sentence numbers. The pipeline then looks up the source text from the real sentences, so an invented quotation is impossible.

Question writing (system role): "You write verification questions for a lecturer to check whether a student genuinely understands and wrote a claim in their own essay. The crucial property: a knowledgeable person who has NOT read this essay should be unable to answer well from general knowledge. So do not ask about general facts; ask the student to reconstruct their own reasoning, name the specific evidence or examples they used, justify the choices they made, and explain how this point connects to the rest of their essay. Avoid yes/no questions and avoid questions that contain their own answer. Reply with JSON only." This wording came out of the Section 8.3 finding that content-naming questions are answerable from general knowledge.

Fine-tune training prompt (identical across v1, v2 and v3, so only the data differs): "You are a teacher. Read the passage and write one clear question that checks whether a student understood it. Reply with the question only."

Corpus generation (system role, used for the original Llama corpus and reused verbatim for the multi-generator test slice): "You are a university student writing a coursework essay. Output only the essay itself as continuous academic prose. Do not write a title, headings, bullet points, a reference list, or any framing such as 'Here is' or 'Sure'. Stay strictly on the given topic and match the academic register of the stated discipline." The user turn anchors the topic with the essay title, keywords extracted from the human source, and the target length.

Appendix B: model and training configurations

| Component | Model | Key settings | |---|---|---| | Detector (transformer) | DeBERTa-v3-base | 3 epochs, lr 2e-5, batch 4 x accum 4, max 512 tokens, fp16, seed 42, student-level splits | | Detector (comparison) | RoBERTa-base | identical settings | | Stylometric model | Gradient boosting | 25 features incl. GPT-2 perplexity; seed 42 | | Hybrid fuser | Logistic regression | over the two model probabilities, fitted on the validation split | | Claim extractor | DeBERTa-v3-base, BIO | paragraph sequences, max 256 subwords, official 322/80 split, strict span scoring | | Relation classifier | DeBERTa-v3-base, pairs | within-paragraph ordered pairs, class-weighted loss, 3 epochs | | Bloom classifier | BERT-base | class weights, stratified 632/135/136, 6 epochs | | QG fine-tunes v1/v2/v3 | Qwen2.5-3B-Instruct | QLoRA: 4-bit NF4, LoRA r=16 on q/k/v/o, 2 epochs, batch 1 x accum 8, paged AdamW 8-bit, prompt-masked loss, 2,600 pairs | | Local generation | Llama 3.1 8B (Ollama) | temperature 0.2, seed 42 | | Embeddings (simulation) | nomic-embed-text | local, via Ollama |

Each experiment writes its full result to a JSON file under `outputs/`. All the numbers quoted in this document trace back to one of those files.

Appendix C: repository map

```
src/
  data/          sampling and corpus construction (BAWE manifest, cleaning)
  generation/    matched AI-essay generation; the multi-generator test slice
  detection/     detector training, audit, normalisation, hybrid fusion, abstain band
  explainability/ Integrated Gradients, SHAP, attention, the per-submission card
  argument_mining/ claim extractor and relation classifier (training and inference)
  question_gen/  prompts, backends (local, commercial, fine-tuned), fine-tunes, gate
  bloom/         Bloom's-level classifier
  evaluation/    discrimination simulation, comparisons, judges, quality audit
  pipeline/      the output assembler (Verification Interview Guide)
outputs/         result JSONs, verification guides (generated)
dissertation/    chapters, figures, decks, meeting records, this document's builder
data/            raw and processed corpora (not committed; licences respected)
models/          trained checkpoints and adapters (not committed)
```

Appendix D: the worked Verification Interview Guide

The complete guide for the worked submission (essay 3108a, AI-generated variant) is included with the electronic submission as `outputs/verification_guides/3108a_ai_guide.pdf`. It is the file the pipeline produced with live models, unedited. Its first page appears as Figure 7.4. The guide contains the hybrid detection summary with component views, the per-submission explanation card, four claims with two-way provenance (cited sentences plus the trained miner's verbatim argument spans), twelve gated verification questions with thinking-level tags, and the three-level suggested rubric for reading the student's answers.